

Table of Contents

[HandlerDefTopic](#)

[SCB68.3.1](#)

[SCB68.3.2](#)

[SCB68.3.3](#)

[hpabout.1.02](#)

[hpabout.1.03](#)

[hpabout.1.04](#)

[hpabout.1.05](#)

[hpabout.1.06](#)

[hpabout.1.07](#)

[hpabout.1.08](#)

[hpabout.1.09](#)

[hpabout.1.10](#)

[hpabout.1.11](#)

[hpabout.1.12](#)

[hpabout.1.13](#)

[hpabout.1.14](#)

[hpabout.1.15](#)

[hpabout.1.16](#)

[hpabout.1.17](#)

[hpfuncts.7.1](#)

[hpfuncts.7.2](#)

[hpgpib.5.1](#)

[hpgpib.5.2](#)

[hpgpib.5.3](#)

[hpgpib.5.4](#)

[hpgpib.5.5](#)

[hpgpib.5.6](#)

[hpgpib.5.7](#)

[hpgpib.5.8](#)

[hpgpib.5.9](#)

[hpp849.8.01](#)

[hpp849.8.02](#)

[hpp849.8.03](#)

[hpp849.8.04](#)

[hpp849.8.05](#)

[hpp849.8.06](#)

[hpp849.8.07](#)

[hpp849.8.08](#)

[hpp849.8.09](#)

[hpp849.8.10](#)

[hpp849.8.11](#)

[hpp849.8.12](#)

[hpp849.8.13](#)

[hpp849.8.14](#)

[hpp849.8.15](#)

[hpp849.8.16](#)

[hpp849.8.17](#)

[hpp849.8.18](#)

[hpp849.8.19](#)

[hpp849.8.20](#)

[hpp849.8.21](#)

[hpparallel.2.1](#)

[hpparallel.2.2](#)

[hpparallel.2.3](#)

[hpparallel.2.4](#)

[hpparallel.2.5](#)

[hpparallel.2.6](#)

[hpparallel.2.7](#)

[hpparallel.2.8](#)

[hprs232.4.01](#)

[hprs232.4.02](#)
[hprs232.4.03](#)
[hprs232.4.04](#)
[hprs232.4.05](#)
[hprs232.4.06](#)
[hprs232.4.07](#)
[hprs232.4.08](#)
[hprs232.4.09](#)
[hprs232.4.10](#)
[hprs232.4.11](#)
[hpsimul](#)

HandlerDefTopic

<

>

About Handler/Prober Communications

About Handler/Prober Communications

This section describes handler/prober communications on IG-XL-based testers, including J750, D750, IP750, FLEX, and UltraFLEX. IG-XL supports parallel, serial, and GPIB tester interfaces. You can also design or develop custom interfaces.

This section assumes experience with IG-XL, the Microsoft Windows environment, and the general functionality of handlers, probers, and testers. A basic understanding of Microsoft Visual Basic use and programming is also a prerequisite. The intended audience is device, production, and software engineers who work with the IG-XL handler/prober interface.

This section includes the following topics:

- [Handler/Prober Communications Overview](#)

- [Operator Interface: EquipmentSelect](#)
- [Operator Interface: TesterControl](#)
- [IG-XL Driver Elements](#)
- [Handler/Prober Driver Flow](#)
- [Driver Installation and Maintenance](#)

The handler/prober document contains the following sections:

- [Parallel Interface](#)
- [RS-232 Interface](#)
- [GPIB Interface](#)
- [IG-XL Simulators](#)
- [Handler/Prober Function Library](#)
- [P849 Communications Interface](#)

Related Documents

Use of handlers and probers often involves test system operators using the IG-XL production environment. For more information, see [About Production Controls](#).

SCB68.3.1

<	>
---	---

Configuring SCB-68 Interconnect Box

Configuring SCB-68 Interconnect Box

This section provides information on the hardware pinout connections from the Interface Controller Card to the SCB-68 Interconnect Box and, finally, to a handler or prober. Signal characteristics (such as timing and levels) are configured through the DIO Ports in the Parallel Interface Driver. The following topics are available:

- [Signals and Pinouts](#)
- [Example Configurations](#)

For more information, see [Parallel Interface](#).

<	>	
	2015 Teradyne, Inc. All rights reserved.	

SCB68.3.2

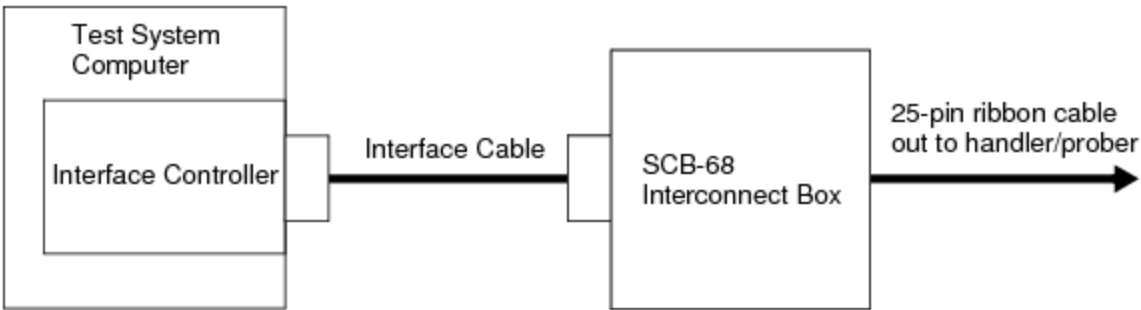
<

>

Configuring SCB-68 Interconnect Box : Signals and Pinouts

Signals and Pinouts

Signals and pinouts from the Interface Controller board are brought out to the SCB-68 Interconnect Box through the interface cable as shown in the figure.



Parallel TTL Interface Components

The SCB-68 box is shipped with a default configuration listed in the table.

Default Pinout Connections of the Interconnect Box

Interconnect Block			
DIO Port	Pin	Function Name	D25 TTL Cable Pin
DIOA0	10	BIN0 MSB	1
DIOA1	44	BIN1	2
DIOA2	45	BIN2	3
DIOA3	12	BIN3	4
DIOA4	13	BIN4	5
DIOA5	47	BIN5	6

DIOA6	48	BIN6	7
DIOA7	15	BIN7	8
DIOB0	16	BIN8	9
DIOB1	17	BIN9	10
DIOB2	51	BIN10	11
DIOB3	52	BIN11 LSB	12
DIOB6	21	END OF TEST (EOS)	13
DIOD0	29	START OF TEST (input)	
DIOD1	63	HANDLER RETEST (input)	15
DIOD2	64	END OF LOT/END OF WAFER (input)	16
DIOD3	31	HANDSHAKE (input)	17
DIOB7	22	HANDLER RETEST (output)	18
DIOD6	67	SPARE	19
DIOD7	34	SPARE	20
DIOD4	32	ALARM	21
+5V	1	+5V SUPPLY	22
DIOD5	66	SPARE	23
GND	36	GROUND	24
GND	37	GROUND	25

Note 1: Pin 1 jumpered to Pin 38 (default) sets data signals as High when tristate.

Note 2: Pin 38 jumpered to Pin 39 (ground) sets data signals as Low when tristate.

Although the table shows signals associated with certain pins, the connections are highly configurable to any application with the exception of these Teradyne reserved signals/pins:

DIOB6	End of Test (EOS/EOT)
DIOD0	Start of Test (SOT)
DIOD1	Retest
DIOD2	End of Lot/End of Wafer
DIOD4	Handshake

A tester can only accept one SOT signal or one combination of SOT/SiteReady line from a handler or prober. So, in the case of multisite testing, there should be SiteReady lines per site and one SOT in parallel for all sites that are ready for test.

The Interconnect Box has extra pins not listed in the table. These extra pins are reserved by the NI-32HS Interface Controller card. Reserved pins match the hard coded GUI driver set up provided with IG-XL.

If more pins are required for a specific handler or prober application, consult with PI Applications engineers on the 64b option kit.

If two SCB-68 boards used, the Teradyne reserved pins only apply to the first board. Equivalent pins on the second board are configurable pins. Reserved lines by the Interface Controller card still apply to both boards.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

SCB68.3.3

<

>

Configuring SCB-68 Interconnect Box : Example Configurations

Example Configurations

If needed, a user manufactured interconnect cable can be connected to the SCB-68 terminal board within the connector block. To assist in wiring, the tables below are provided as an example. One application may work using only one SCB-68 board for up to eight sites, where another application may require two SCB-68 boards depending on the type of handler or prober used. Refer to the following tables for example configurations:

- Single/Dual-Site SCB-68 Connections
- Quad-Site SCB-68 Connections, Board 1 of Two
- Quad-Site SCB-68 Connections, Board 2 of Two
- Octal-Site SCB-68 Connections, Board 1 of Two
- Octal-Site SCB-68 Connections, Board 2 of Two

In the tables, pins in courier text are Interface Controller reserved lines and pins in bold text are Teradyne reserved lines.

Single/Dual-Site SCB-68 Connections

Pin	Signal	Pin	Signal
1	+5 volts	35	RGND

2	REQ1	36	GND
3	ACK1 (STARTTRIG1)	37	GND
4	STOPTRIG1	38	+5 Selected
5	PCLK1	39	GND
6	PCLK2	40	CPULL
7	STOPTRIG2	41	GND
8	ACK2 (STARTTRIG2)	42	GND
9	REQ2	43	RGND
10	Bin 0 Site 0 (LSB)	44	Bin 1 Site 0
11	GND	45	Bin 2 Site 0
12	Bin 3 Site 0	46	GND
13	Bin 4 Site 0	47	Bin 5 Site 0
14	GND	48	Bin 6 Site 0
15	Bin 7 Site 0	49	GND
16	Bin 8 Site 0	50	GND
17	Bin 9 Site 0	51	Bin 10 Site 0
18	GND	52	Bin 11 Site 0 (MSB)
19	RGND	53	Bin 0 Site 1 (LSB)
20	GND	54	Bin 1 Site 1
21	END OF SITE 1	55	GND
22	END OF SITE 2	56	RGND
23	Bin 2 Site 1	57	Bin 3 Site 1
24	GND	58	Bin 4 Site 1
25	Bin 5 Site 1	59	GND
26	Bin 6 Site 1	60	Bin 7 Site 1
27	GND	61	Bin 8 Site 1
28	Bin 9 Site 1 (MSB)	62	GND
29	Start of Test (Parallel)	63	Handler Retest (IN)
30	GND	64	EOL/EOW (IN)

31	Handshake (IN)	65	GND
32	Handler Alarm	66	Site Ready 0
33	GND	67	Site Ready 1
34	Spare	68	GND

Quad-Site SCB-68 Connections, Board 1 of Two

Pin	Signal	Pin	Signal
1	+5 volts	35	
2	REQ1	36	
3	ACK1 (STARTTRIG1)	37	
4	STOPTRIG1	38	
5	PCLK1	39	
6	PCLK2	40	
7	STOPTRIG2	41	
8	ACK2 (STARTTRIG2)	42	
9	REQ2	43	
10	Bin D0 Site 1 (LSB)	44	Bin D1 Site 1
11		45	Bin D2 Site 1
12	Bin D3 Site 1	46	
13	Bin D4 Site 1 (MSB)	47	Bin D0 Site 2 (LSB)
14		48	Bin D1 Site 2
15	Bin D2 Site 2	49	
16	Bin D3 Site 3	50	
17	Bin D4 Site 3 (MSB)	51	
18		52	
19		53	
20		54	
21	End of Site 1	55	

22	End of Site 2	56	
23		57	
24		58	
25		59	
26		60	
27		61	
28		62	
29	Start of Test (Parallel)	63	
30		64	
31	Site 1 Ready	65	
32		66	Site 2 Ready
33		67	Site 3 Ready
34	Site 4 Ready	68	

Quad-Site SCB-68 Connections, Board 2 of Two

Pin	Signal	Pin	Signal
1	+5 volts	35	
2	REQ1	36	
3	ACK1 (STARTTRIG1)	37	
4	STOPTRIG1	38	
5	PCLK1	39	
6	PCLK2	40	
7	STOPTRIG2	41	
8	ACK2 (STARTTRIG2)	42	
9	REQ2	43	
10	Bin D0 Site 3 (LSB)	44	Bin D1 Site 3
11		45	Bin D2 Site 3
12	Bin D3 Site 3	46	

13	Bin D4 Site 3 (MSB)	47	Bin D0 Site 4 (LSB)
14		48	Bin D1 Site 4
15	Bin D2 Site 4	49	
16	Bin D3 Site 4	50	
17	Bin D4 Site 4 (MSB)	51	
18		52	
19		53	
20		54	
21	End of Site 3	55	
22	End of Site 4	56	
23		57	
24		58	
25		59	
26		60	
27		61	
28		62	
29		63	
30		64	
31		65	
32		66	
33		67	
34		68	

Octal-Site SCB-68 Connections, Board 1 of Two

Pin	Signal	Pin	Signal
1	+5 volts	35	
2	REQ1	36	
3	ACK1 (STARTTRIG1)	37	

4	STOPTRIG1	38	
5	PCLK1	39	
6	PCLK2	40	
7	STOPTRIG2	41	
8	ACK2 (STARTTRIG2)	42	
9	REQ2	43	
10	Bin 0 Site 1 (LSB)	44	Bin 1 Site 1
11		45	Bin 2 Site 1
12	Bin 3 Site 1 (MSB)	46	
13	Bin 0 Site 2 (LSB)	47	Bin 1 Site 2
14		48	Bin 2 Site 2
15	Bin 3 Site 2 (MSB)	49	
16	Bin 0 Site 3 (LSB)	50	
17	Bin 1 Site 3	51	Bin 2 Site 3
18		52	Bin 3 Site 3 (MSB)
19		53	End of Site 1
20		54	End of Site 2
21	End of Site 3	55	
22	Site 8 Disable	56	
23	Not Used	57	Not Used
24		58	Not Used
25	Not Used	59	
26	Site 5 Ready	60	Site 6 Ready
27		61	Site 7 Ready
28	Site 8 Ready	62	
29	Start of Test (Parallel)	63	EOL
30		64	Retest
31	Site 1 Ready	65	
32	Alarm	66	Site 2 Ready

33		67	Site 3 Ready
34	Site 4 Ready	68	

Octal-Site SCB-68 Connections, Board 2 of Two

Pin	Signal	Pin	Signal
1	+5 volts	35	
2	REQ1	36	
3	ACK1 (STARTTRIG1)	37	
4	STOPTRIG1	38	
5	PCLK1	39	
6	PCLK2	40	
7	STOPTRIG2	41	
8	ACK2 (STARTTRIG2)	42	
9	REQ2	43	
10	Bin 0 Site 4 (LSB)	44	Bin 1 Site 4
11		45	Bin 2 Site 4
12	Bin 3 Site 4	46	
13	Bin 0 Site 5 (LSB)	47	Bin 1 Site 5
14		48	Bin 2 Site 5
15	Bin 3 Site 5 (MSB)	49	
16	Bin 0 Site 6 (LSB)	50	
17	Bin 1 Site 6	51	Bin 2 Site 6
18		52	Bin 3 Site 6 (MSB)
19		53	Bin 0 Site 7 (LSB)
20		54	Bin 1 Site 7
21	Bin 2 Site 7	55	
22	Bin 3 Site 7 (MSB)	56	
23	Bin 0 Site 2 (LSB)	57	Bin 1 Site 2

24		58	Bin 2 Site 2
25	Bin 3 Site 2 (MSB)	59	
26	End of Site 4	60	End of Site 5
27		61	End of Site 6
28	End of Site 7	62	
29	End of Site 8	63	Site 1 Disable
30		64	Site 2 Disable
31	Site 3 Disable	65	
32	Site 4 Disable	66	Site 5 Disable
33		67	Site 6 Disable
34	Site 7 Disable	68	

hpabout.1.02

	
---	---

About Handler/Prober Communications : Handler/Prober Communications Overview

Handler/Prober Communications Overview

Handler/prober communications are a part of the production interface on the tester. Typically, each device that is tested is cycled by the communications between material handling equipment (handler or prober) and the tester. While the tester can test parts independently and the handler or prober can cycle parts independently, the communications link is what creates a test cell that works together. Therefore, the entire production interface (including mechanical, signal delivery DIB/HIB/PIB, communications, and production software) must function correctly. Communications is an important link in this process.

Handler/prober communications are important to a test cell because the communications link is the only link between the tester and a handler or prober. The communications interface must automate the test cycle while maximizing throughput and minimizing latency. This interface must also provide status and lot information to the tester to aid in data collection and test cell monitoring.

Many models and types of handlers are available, while only a few types of probers exist. Handlers, which are designed for high throughput of packaged devices, come in different designs for different package types. Device package types can range from the traditional dual in-line package (DIP) to strip devices. Strip devices are interconnected and are tested in a group like a wafer. Regardless of device type, handlers are designed for packaged final test where the devices are ready to be sold or used.

Probers, on the other hand, are used for high accuracy testing of devices in wafer form. Probers are methodical and slower than handlers. A prober's job, however, is to make sure that the probe needles on the probe card always contact the bonding pads on the devices. Speed is secondary to accuracy.

Both handlers and probers must communicate with the tester to indicate when they have product to test. The tester must communicate with the handler or prober to relay the results of each test.

Probers must also send the X-Y location of the device on the wafer to the tester in addition to the test start.

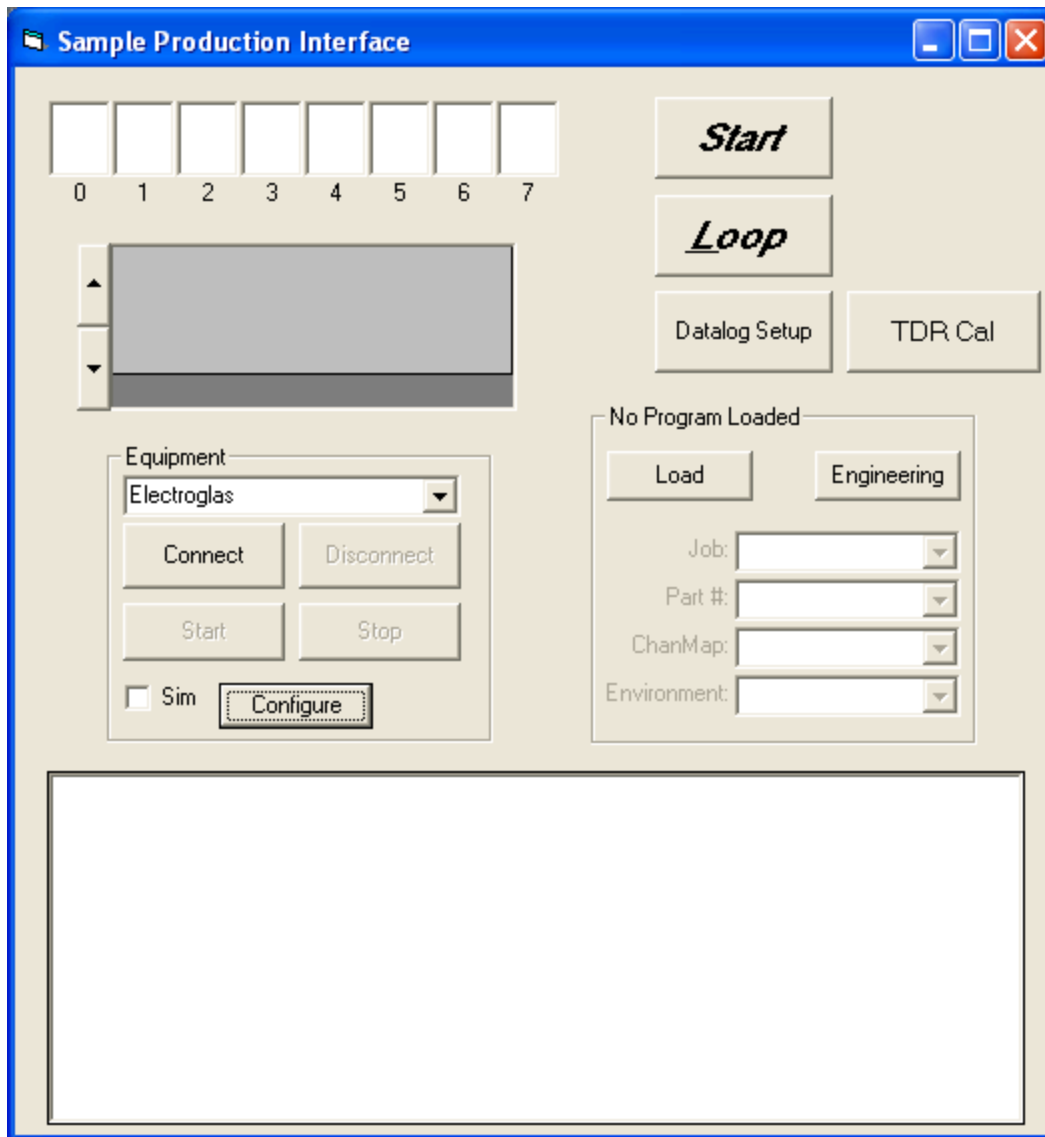
Therefore, handlers and probers work differently internally and have different communications needs. Most handlers and probers conform to some communications standards, however, greatly simplifying the problem. Still, since handlers and probers are manufactured by different companies, the tester must be flexible enough to communicate with all of them while keeping the overall cycle time low. So, to run production efficiently, you must understand the handler/prober communications aspect of production.

IG-XL Overview

IG-XL-based testers have an interface that allows you to customize communications, including what data is transferred and in what order. This interface allows you to focus on user-level communications instead on the hardware-level software drivers.

IG-XL also allows you to customize the operator interface (OI) and the user interface (UI). An OI is used by production floor personnel to set up, configure, and operate the tester. Most if not all functions that an operator performs should be part of the OI. These include test program loading and unloading, handler and prober connections, and datalogging.

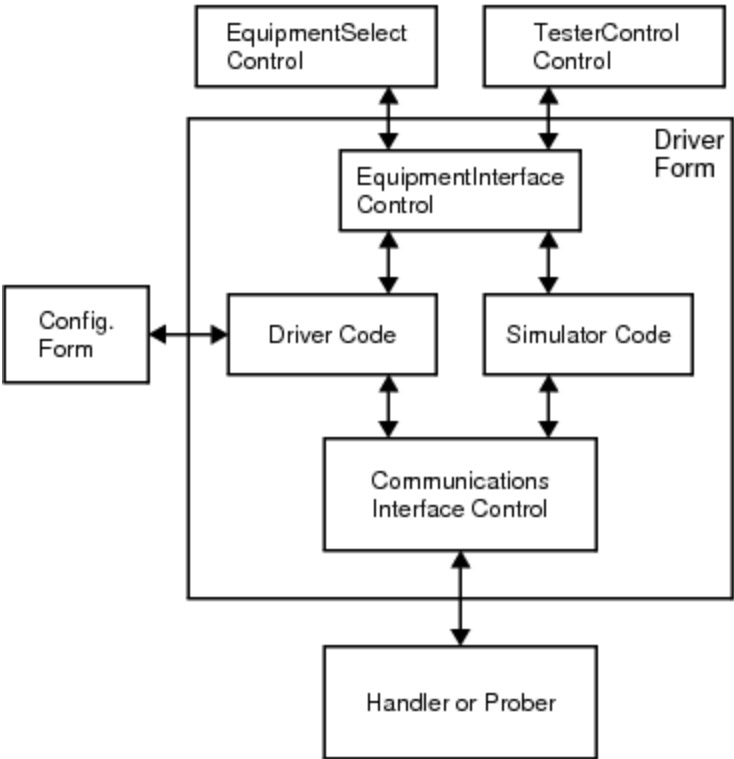
IG-XL includes three interfaces (SimpleOI, CustomOI, and ProductionUI) that can either serve as examples or can themselves be used as the tester OI. Each OI, whether custom or standard, is created using IG-XL objects on a Visual Basic form. Since SimpleOI is uncomplicated and performs all the basic functions, it is used in this section when discussing the OI.



SimpleOI Operator Interface

IG-XL is a Microsoft Windows-based product. Therefore, software drivers for IG-XL-based testers are written in Microsoft's Visual Basic language. This allows you to use standard Windows displays and interfaces. Handler/prober communications on IG-XL-based testers are independent dynamic link libraries (DLLs) that are compiled and registered in the Windows registry and can be transported from tester to tester.

Each IG-XL handler/prober driver interfaces with the tester in the same way. You write custom code that communicates with standard IG-XL control objects. Little code is required other than the code that is necessary to parse and interpret responses from the handler or prober. This figure provides a functional diagram for the way that the driver interacts with the rest of the tester and with the handler or prober.



IG-XL Handler/Prober Driver Interaction

Outside the driver are the external elements that it must interact with (EquipmentSelect, TesterControl, Handler/Prober) and use (Config form). Essentially, the driver is the conduit between controls in the tester’s OI and the handler or prober. Internal to the driver, the EquipmentInterface control takes stimulus from the tester which sends it for processing inside the driver which in turn passes the processed information to be output on a hardware interface to the handler or prober. Thus, you must understand the OI interaction with the driver as well as the internal functionality of the driver. [Operator Interface: EquipmentSelect](#) and [Operator Interface: TesterControl](#) explain the critical OI elements that the driver interacts with. [IG-XL Driver Elements](#) explains the inner structure and functions of a driver.

hpabout.1.03

<	>
-----------------	-----------------

About Handler/Prober Communications : Operator Interface: EquipmentSelect

Operator Interface: EquipmentSelect

The EquipmentSelect object is included on most OIs and allows the OI to interact with the driver. The EquipmentSelect object, part of the IG-XL ProductionControls.ocx library, includes a fully-developed GUI along with relevant properties, events, and methods. Using the EquipmentSelect object, the OI can use any driver without having any communications code inside the driver. The following topics are available:

- | | |
|---|----------------|
| • | User Interface |
| • | Properties |
| • | Events |
| • | Methods |

For more information, see [EquipmentSelect Control](#) in the Production Controls section.

<	>
-----------------	-----------------

hpabout.1.04

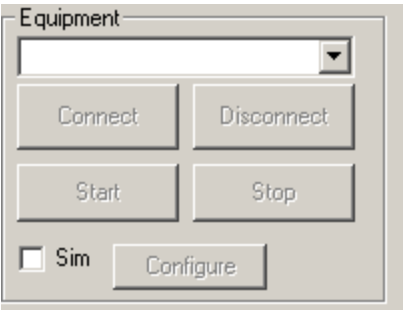
<

>

About Handler/Prober Communications : Operator Interface: EquipmentSelect : User Interface

User Interface

The EquipmentSelect has a fully developed user interface. Using the EquipmentSelect, the OI can find, create, and destroy equipment drivers programmatically and retrieve handles to the driver objects and communicate directly with the equipment when necessary. Each driver that has been registered can then be selected and configured based on its design using the EquipmentSelect object. Since each driver can be completely customized, each can be different. However, some common elements in every driver must be present. The standard EquipmentSelect object is shown in the figure.



EquipmentSelect IG-XL Visual Basic Object

Using the EquipmentSelect GUI, test system operators can view the list of available drivers, select one, and connect, start, stop, disconnect, and configure the equipment. For offline development, a checkbox turns on equipment simulation mode.

The EquipmentSelect object is used in an OI to select and use a handler/prober driver that is installed on the tester. Each EquipmentSelect has the following elements that each perform a specific function in the driver:

- | | |
|---|---------|
| • | Name |
| • | Connect |

- [Disconnect](#)

- [Start](#)

- [Stop](#)

- [Sim](#)

- [Configure](#)

Each driver must also contain these elements to function. The functionality of the driver otherwise is completely optional as long as these high-level elements are present.

Name

Each driver must be given a name when it is compiled. The driver name (the name of the DLL file) is reflected in the EquipmentSelect menu. When a registered driver is selected from the menu, it is loaded and can then be used by the test system on Connecting and Starting. Selecting the Name is always the first operation when loading a handler/prober driver for use in production using the EquipmentSelect object.

Nothing is required (other than a name) in the driver, but the driver must be registered in the Windows registry for the EquipmentSelect object to recognize it.

For more information, see [Driver Installation and Maintenance](#).

Connect

The Connect element performs any initialization that is required for the handler or prober before the test cycle begins. In general, any communication, setup, or configuration that is special or nonstandard for testing (first-run/run-once or initialization routines) to start is included. Therefore, there are no required operations when connecting. The Connect() function is always the second operation when using a handler/prober driver through the EquipmentSelect object.

When you click the Connect button, the EquipmentSelect object loads a chain of initialization functions ending in the Connect() function of the driver. The Connect() function must be present (but may be empty) for the driver to operate regardless of the contents.

For more information, see [Handler/Prober Driver Flow](#).

Disconnect

The Disconnect element performs any steps that are required to end or abort the test cycle.

Typically, a handler or prober can be stopped in production at any point. If any post-production communications or setups must be done on the handler or prober, however, then the Disconnect()

function is the proper place to include code. The Disconnect() function is usually the last function to be executed but is not enforced by IG-XL.

When you click the Disconnect button, the EquipmentSelect object executes the Disconnect() function in the driver. Like Connect(), no code is required but the function must be present. Since the Disconnect function is not required, note that any code placed in Disconnect() may not be executed since the operator can usually quit the OI without disconnecting the handler or prober.

Start

The Start element begins the test cycle. Typically, this involves polling the handler or prober for a test start, executing the test program, and reporting the bin result to the handler or prober. A timer that is part of the OI typically controls the polling. Start is the last function that must be executed before production starts.

When you click the Start button, the EquipmentSelect object executes the StartPolling() function that activates the timer to alarm at the set polling rate. The handler or prober is polled and the test program is run with any other necessary communications in between. After the test program runs, the driver is re-entered (by the TesterControl control) at the TestComplete() function that sends the binning to the handler or prober, finishes the test cycle, and prepares to begin the next one by re-enabling the poll timer.

For more information, see [Operator Interface: TesterControl](#).

Stop

The Stop element ends the test cycle after the current iteration. The driver is still loaded and testing can continue again you click if the Start button. No code is typically required for the Stop function other than stopping the poll cycle, but you can add any custom code that may be necessary.

When you click the Stop button, the EquipmentSelect object executes the StopPolling() function. The StopPolling() function must exist in the driver and typically contains a call to disable the poll timer.

Sim

Writing a handler/prober simulator is useful to debug and verify the interface. This allows the driver to be tested or exhibited when it is not connected to the handler or prober. When the Sim checkbox is checked or unchecked, this enables or disables the simulator mode of the driver. If a simulator has been written for the handler/prober driver for an offline demo, it will participate in the test cycle instead of the handler or prober. The tester (and the driver) will not know that it is not actually communicating with a piece of external equipment.

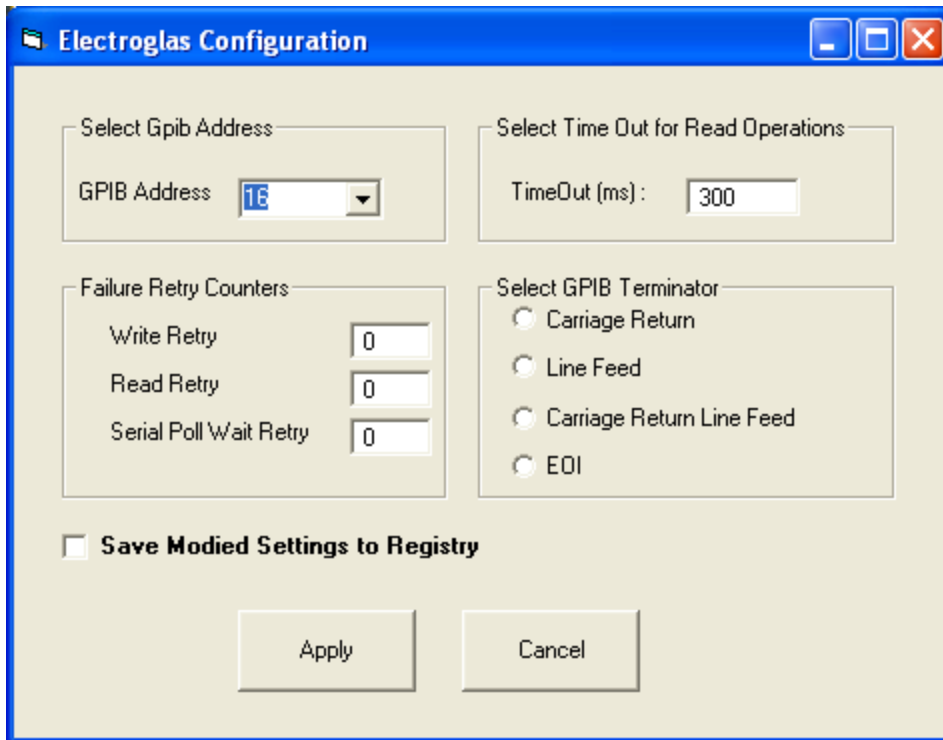
When the Sim checkbox status changes, the SetSimulatorMode() function is called by the EquipmentSelect object. The SetSimulatorMode() function typically only processes the setting that was made. In addition, the Sim checkbox can only be set while the handler or prober is selected but not Connected or Started.

For more information, see [IG-XL Simulators](#).

Configure

The Configure element causes a separate form to load up that allows you to change a driver's operation so that it will execute in a particular configuration. Most handlers and probers have settings that alter the way that their interface responds. It is useful, therefore, to make the driver configurable so that it can work in all modes.

When you click the Configure button, the EquipmentSelect object loads a form that has all the programmed settings that can also be saved in the registry. The EquipmentSelect object calls the Config() function in the driver to load up the form that will have code of its own to manage the settings process. This figure shows a typical Configure form.



The image shows a Windows-style dialog box titled "Electroglas Configuration". It has a blue title bar with standard minimize, maximize, and close buttons. The dialog is divided into several sections:

- Select Gpib Address:** A group box containing a "GPiB Address" label and a dropdown menu showing the value "16".
- Select Time Out for Read Operations:** A group box containing a "TimeOut (ms) :" label and a text input field containing the value "300".
- Failure Retry Counters:** A group box containing three labels and text input fields: "Write Retry" (0), "Read Retry" (0), and "Serial Poll Wait Retry" (0).
- Select GPIB Terminator:** A group box containing four radio button options: "Carriage Return", "Line Feed", "Carriage Return Line Feed", and "EOI".
- Save Modied Settings to Registry:** A checkbox that is currently unchecked.
- Buttons:** At the bottom, there are two buttons labeled "Apply" and "Cancel".

Configure Form for Electroglas GPIB Interface

		
	2015 Teradyne, Inc. All rights reserved.	

hpabout.1.05

<

>

About Handler/Prober Communications : Operator Interface: EquipmentSelect : Properties

Properties

The table lists the properties for the EquipmentSelect control object. The Read/Write status of each property is listed in the two rightmost columns. Properties that are read only can only be read by the driver. Properties that are both Read and Write can be altered.

Properties for EquipmentSelect Control Object

Name	Type	Description	Design Time	Run Time
CurrentDriver	String	Stores the text name of the driver that is currently selected.	N/A	R
Driver	Object	Handle to the currently selected handler/prober driver, if any (an EquipmentInterface object). Provides access to properties and methods of the standard interface. When assigned to a variable declared WithEvents, also allows trapping handler driver events.	N/A	R
DriverCustom	Object	Handle to the currently selected handler/prober driver's class instance. Allows access to custom properties and methods in the class.	N/A	R
SimVisible	Boolean	Whether to add a simulation mode checkbox to the GUI. If true, add the	R/W	R/W

checkbox to select simulator mode. If false, the Simulator property of the driver's HandlerTesterControl specifies the mode. Only use this control during development. Remove it from the production version to prevent simulation mode from being used in production.

The properties for the EquipmentSelect are not as widely used as the properties for the objects in the driver. However, the DriverCustom property is often useful. To allow the OI access to information that the driver may have internally (such as device ID, temperature data, and alarms), the DriverCustom is the expansion point for all custom functions that are not included as IG-XL properties, events, or methods. For more information on expanding a driver's functionality using DriverCustom, see EquipmentInterface Control.

<		>	
		2015 Teradyne, Inc. All rights reserved.	

hpabout.1.06

<

>

About Handler/Prober Communications : Operator Interface: EquipmentSelect : Events

Events

The EquipmentSelect events are centered on extracting critical information from the handler or prober into the OI. These events are product or equipment status that may be important for the OI to act on.

For example, OI designers may wish the datalog to automatically close at the end of a prober’s cassette. By including an EndOfCassette() callback function in the OI code, the EquipmentSelect event can trigger the desired behavior. If the event occurs but no function is present in the OI, an error message is displayed but testing continues. For example, if the driver processes an End Of Wafer event but no EndOfWafer function is available, then the tester places a popup window warning that the End Of Wafer occurred but there was nothing to execute. This table lists the events for the EquipmentSelect control object.

Events for EquipmentSelect Control Object

Name	Arguments	Description
Disconnected	(none)	Event raised when an Equipment Driver is disconnected.
EndOfCassette	(none)	Event raised when a handler or prober reaches the end of cassette (EOC) and the driver asserts EOC. The EndOfCassette() function is called.
EndOfWafer	(none)	Event raised when a handler or prober reaches the end of wafer (EOW) and the driver asserts EOW. The EndOfWafer() function is called.

EquipmentSelected	NewDriver As Object	Driver has been changed. If the client maintains a pointer to the driver, the pointer should be updated from the argument. When a driver is destroyed, this event is also raised, with an argument of "Nothing".
ErrorDetected	code As Integer mesg As String	Event raised when an error is detected in the Equipment Driver. The error code and message is passed in.
OutOfProduct	(none)	Event raised when a handler or prober runs out of product and the driver asserts OutOfProduct. The OutOfProduct() function is called.
PollingDisabled	(none)	Event raised when an Equipment Driver is told to stop polling for starts, calling the StopPolling() function.
PollingEnabled	(none)	Event raised when an Equipment Driver is told to start polling for starts, calling the StartPolling() function.
StartOfTest	siteMask As Long xcoord As Long ycoord As Long	Event raised when a test start from the handler or prober is received.
StartofWafer	(none)	Event raised when a handler or prober reaches the start of wafer (SOW) and the driver asserts SOW. The StartOfWafer() function is called.

Note about EquipmentSelect events that the EquipmentInterface (see EquipmentInterface Control) has similar Events that trigger functions with the same names. In the case where the driver (EquipmentInterface) and the OI (EquipmentSelect) both have a function, both functions execute. For example, if both the driver and the OI have implemented EndOfWafer() functions and an EndOfWafer event occurs, both EndOfWafer() functions execute.

☑ Note:

The driver's event triggers its function first, and then the OI event triggers its function afterwards. The handler/prober driver's methods originate the events that call the driver or OI callback functions. See Methods.

For the normal case where either the driver or OI does not have a function, the module that has the function executes its function. Place the callback functions in the OI instead of in the driver so that

the same driver can be used on multiple systems that may or may not have the need for the functionality. Most Teradyne-supplied drivers do not include callback functions for this reason.

< >	
	2015 Teradyne, Inc. All rights reserved.

hpabout.1.07

<

>

About Handler/Prober Communications : Operator Interface: EquipmentSelect : Methods

Methods

Methods for the EquipmentSelect are strictly for managing what driver is currently selected. In most cases, the system manages this through the EquipmentSelect object’s button. To have the OI perform EquipmentSelect tasks (for instance, so that the EquipmentSelect is not used by the operator), use these functions. Most applications do not require this. This table lists methods for the EquipmentSelect control object.

Methods for EquipmentSelect Control Object			
Name	Type	Arguments	Description
DestroyDriver	N/A	(none)	Stops, disconnects, and destroys the currently selected equipment driver. Raises the EquipmentSelected event with the argument "Nothing".
GetRegisteredDrivers	N/A	DriverList() As String, numDrivers As Integer	Searches the system registry and returns the list of available handler drivers. A string from the list can be passed to SelectDriver().
SelectDriver	N/A	DrvName As String	Creates a new equipment driver object. Previous driver, if any, is destroyed. Raises the EquipmentSelected event if successful.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

hpabout.1.08

<	>
-----------------	-----------------

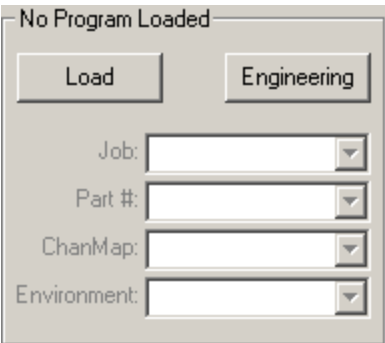
About Handler/Prober Communications : Operator Interface: TesterControl

Operator Interface: TesterControl

Like the EquipmentSelect control, the TesterControl control is an object that is part of the OI. While the EquipmentSelect control is focused on control and operation of the IG-XL handler/prober driver, the TesterControl control is actually used by the driver.

The TesterControl control’s primary function is to control and operate the test program, a function critical to the handler/prober driver. This topic focuses on how the TesterControl control is used by the IG-XL handler/prober driver. Only IG-XL handler/prober driver relevant details on the TesterControl are described here. Note that the TesterControl control is always found on the OI and not in the driver.

The TesterControl control is shown in the figure.



TesterControl IG-XL Visual Basic Object

The following topics are available:

- | | |
|---|----------------------------|
| • | Properties |
| • | Events |
| • | Methods |

For more information, see [TesterControl Control](#) in the Production Controls section.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hpabout.1.09

<

>

About Handler/Prober Communications : Operator Interface: TesterControl : Properties

Properties

The TesterControl control has several properties that may be useful to a handler/prober driver. The table lists these properties. Note that all TesterControl properties are not listed but are described elsewhere in IG-XL Help. TesterControl properties are used indirectly in handler/prober drivers through the EquipmentInterface (on the driver’s form) using the TesterHandle method (see [Selected Methods for TesterControl Control Object](#)). See [Methods](#) for a description of the EquipmentInterface.

Selected Properties for TesterControl Control Object

Name	Type	Description	Design Time	Run Time
CurrentEnv	String	Name of the active environment in the loaded test program.	N/A	R/W
CurrentJob	String	Name of the active job in the loaded test program.	N/A	R/W
CurrentPart	String	Name of the active part in the loaded test program.	N/A	R/W
ProgramLoaded	Boolean	Whether a test program is currently loaded.	N/A	R
Sites	Integer	Number of sites in the loaded program, or system maximum if no program is loaded.	N/A	R

The property that is most useful is the Sites property. Using the Sites property in a driver can allow the driver to adapt to the currently loaded test program without requiring you to specify how many sites the test program supports. In many cases, knowing this information in real time can increase efficiency by tailoring the driver to the requirements instead of creating a driver that is less efficient for the general case.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hpabout.1.10

<

>

About Handler/Prober Communications : Operator Interface: TesterControl : Events

Events

The events for the TesterControl control are not as useful to a driver since the event occurs where the TesterControl resides, in the OI. A few TesterControl events are useful, however, because they affect the driver. (See the table below for pertinent handler/prober driver events.) The most notable is the TestComplete event. The TestComplete event is what completes the test cycle by calling the TestComplete() function of the driver (see Handler/Prober Driver Flow). TesterControl events are not triggerable by the driver directly, but they do affect the same test cycle flow.

An added feature of the TestComplete functionality is that a test program is not required to simulate binning. If the tester receives a test start without a loaded program, simulated binning is returned to the handler or prober. This can be useful when debugging a handler/prober driver where loading a test program would otherwise be required.

Selected Events for TesterControl Control Object

Name	Arguments	Description
ProgramLoad	Loaded As Boolean	A program has been loaded or unloaded.
SitesChange	TotalSites As Integer, ActiveSites() As Boolean	The list of active sites has changed. For each site in TotalSites, ActiveSites array is true if that site is active.
TestComplete	HardBin() As Long, SoftBin() As Long, Result() As Long	A test program execution has completed. The arguments report the results as arrays per site: bin numbers (-1 for site not tested), sort numbers (-1 for site not tested), and Pass/Fail result (1 = pass, 2 = fail, -1 = not tested - the constants FPASS and

FFAIL can be used for the pass and fail values). Calls the driver's TestComplete() function.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hpabout.1.11

<

>

About Handler/Prober Communications : Operator Interface: TesterControl : Methods

Methods

Since the TesterControl interacts with the tester and the driver, several methods are of interest for handler/prober drivers. The table shows a selected list of the available methods. Like the properties, the TesterControl methods are used indirectly through the EquipmentInterface (on the driver’s form) using the TesterHandle construct.

Selected Methods for TesterControl Control Object

Name	Type	Arguments	Description
DisableAllSites	N/A	(none)	Deactivates all test sites.
DisableSite	N/A	Site As Integer	Deactivates a specific site. Sites are numbered starting from zero.
EnableAllSites	N/A	(none)	Activates all test sites.
EnableSite	N/A	Site As Integer	Activates a specific site. Sites are numbered starting from zero.
EnableSiteForRetest	N/A	Site As Integer, [datalogOn As Boolean]	Selects the specified site for retesting its current device. Optionally can datalog the retest or not.
GetBin	Long	Site As Integer	Gets the last bin result for the given site, or –1 for not tested.
GetResult	Long	Site As Integer	Returns the last pass/fail result for a specific site: pass = 1, fail = 0, not

tested = -1.

GetSites	Long	ActiveSites() As Boolean	Gets the list of active sites. ActiveSites is indexed by site number and is true if that site is enabled. Returns number of sites in ActiveSites (matches number of sites in the active Chanmap, or system maximum if no program is loaded).
GetSort	Long	Site As Integer	Gets the last sort number for the given site, or -1 if the site was not tested.
StartTest	Boolean	(none)	Executes the currently loaded test program, or returns simulated binning if no test program is loaded. The test results are reported by a TestComplete event.
TesterHandle	N/A	Object	Allows external DLL handler/prober drivers to access the TesterControl properties and methods.

For a description of the EquipmentInterface, see Methods.

All handler/prober drivers use at least one call to the TesterControl control. To make the test program execute as part of the test cycle, the TesterControl’s StartTest method is called using the TesterHandle construct. Note that most of the other methods are called by EquipmentInterface functions (for example, setting the site mask information) but are useful for reference.

hpabout.1.12

<	>
----------------------	----------------------

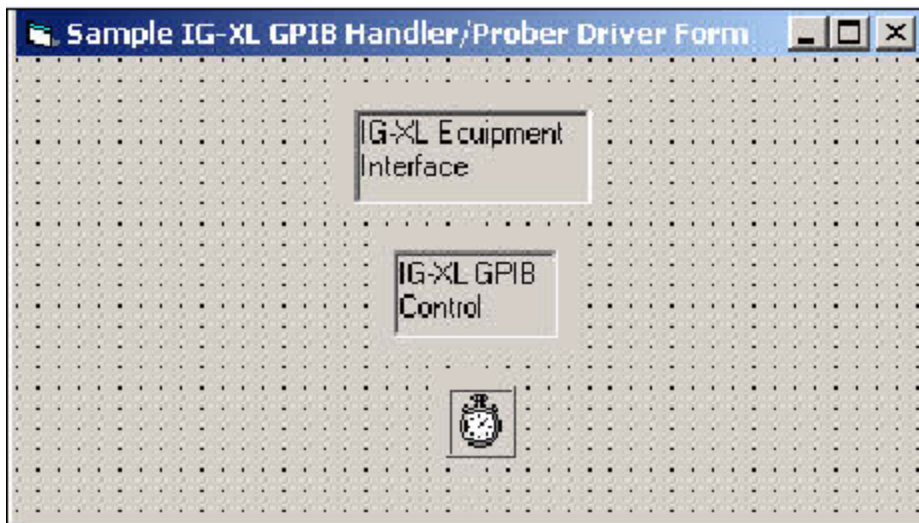
About Handler/Prober Communications : IG-XL Driver Elements

IG-XL Driver Elements

Each IG-XL handler/prober driver consists of three elements:

- [EquipmentInterface Control](#)
- [Communications Control](#)
- [Communications Timer](#)

These elements are normally present on the form for the handler/prober driver. The form, not typically displayed to the tester's user, stores all the code for a driver. This figure shows an example of a handler/prober driver's VB form.



IG-XL Handler/Prober Driver Form Example

All three elements are on the form (as well as the code that drives them) and are accessible. These elements each have key properties, events, and methods that are useful for creating a

[handler/prober driver.](#)

<	>	
	2015 Teradyne, Inc. All rights reserved.	

hpabout.1.13

<

>

About Handler/Prober Communications : IG-XL Driver Elements : EquipmentInterface Control

EquipmentInterface Control

IG-XL has a standard set of objects that can be built into the OI or handler/prober driver forms to bring preset functionality in. For handler/prober drivers, the main object is the IG-XL EquipmentInterface control. The EquipmentInterface is a control that allows you to build a custom driver around a common skeleton.

The EquipmentInterface is the conduit for information and control between an operator interface and a handler/prober driver. The existence of the EquipmentInterface control on the form is what makes the form a handler/prober driver. Each handler/prober driver, therefore, uses the same EquipmentInterface control. The code that is written differs based on the interfacing needs (such as how a handler or prober manufacturer has designed the interface). However, a standard set of properties, events, and methods are available to each handler/prober driver to create some uniformity.

For more information, see EquipmentInterface Control in the Production Controls section.

Properties

The table lists properties for the EquipmentInterface control object. The Read/Write status of each property is listed in the two rightmost columns. Properties that are read only can only be read by the driver, whereas properties that are both Read and Write can be altered.

Properties for EquipmentInterface Control Object

Name	Type	Description	Design Time	Run Time
Connected	Boolean	Whether the selected equipment driver is connected (communication established).	N/A	R

DriverState	See note	The state of the selected equipment driver. Type is EquipmentDriverState; possible values are hifConnected, hifDisconnected, hifPaused, hifPolling.	N/A	R
Enabled	Boolean	Whether the selected equipment driver is polling for starts.	N/A	R
ErrorCode	Integer	Equipment-specific error code from the last ErrorDetected event.	N/A	R
ErrorMessage	Variant	Equipment-specific error message describing ErrorCode.	N/A	R
ID	String	Equipment identification string.	N/A	R
SimulatorMode	Boolean	Whether driver is in offline simulation mode.	N/A	R/W
TestSiteTemperature	Double	Test site temperature in degrees C.	N/A	R
WaferAutoSite	Boolean	For probers: when true, setting site 0 wafer X or Y automatically sets all other sites based on WaferXSiteInc and WaferYSiteInc values.	N/A	R/W
WaferID	String	The current wafer ID.	N/A	R
WaferSiteDeltaX(Site As Long)	Long	For probers: amount to increment X position for each site when WaferAutoSite is true. Default: 1	N/A	R/W
WaferSiteDeltaY(Site As Long)	Long	For probers: amount to increment Y position for each site when WaferAutoSite is true. Default: 0	N/A	R/W
WaferSiteKey	Long	The primary site for the current wafer. All deltas are measured from the primary site.	N/A	R/W

EquipmentInterface properties are primarily read-only with a few exceptions. Normally, the properties are used to monitor the status of the driver and get information about the wafer under test. The intent is not to store information about the devices under test but to monitor the interface. All device-specific information is usually found in-line throughout the driver code in local or global variables. Thus, information that is available or collected for a particular driver is completely custom.

Events

Unlike properties for the `EquipmentInterface`, events and methods are centered on device-specific information. Physical events that occur on a handler or prober that are communicated to the tester can in turn trigger methods that trigger events that call a particular callback function. Thus, `EquipmentInterface` events are conduits for calling custom callback code (for instance, `EndOfWafer()`) that is in the driver.

The table lists events for the `EquipmentInterface` control object.

Events for `EquipmentInterface` Control Object

Name	Arguments	Description
Connected	(none)	Connect method has executed successfully.
Disconnected	(none)	Disconnect method has executed successfully.
EndOfCassette	(none)	For probers: all wafers in the cassette have been tested.
EndOfWafer	(none)	For probers: all devices on a wafer have been tested.
ErrorDetected	code As Integer mesg As String	An error has occurred, available to be read from the <code>ErrorCode</code> and <code>ErrorMessage</code> properties.
OutOfProduct	(none)	For handlers: no more devices available.
PollingDisabled	(none)	Driver has stopped polling for starts.
PollingEnabled	(none)	Driver has begun polling for test start signals.
StartOfTest	(none)	Driver has received a start of test signal.
StartOfWafer	(none)	For probers: A new wafer is being tested.

Note for `EquipmentInterface` events that the `EquipmentSelect` (see [Events](#)) has similar events that trigger functions with the same names. In the case where the driver (`EquipmentInterface`) and the OI (`EquipmentSelect`) both have a function, both functions execute. For example, if both the driver and the OI have implemented `EndOfWafer()` functions and an `EndOfWafer` event occurs, both `EndOfWafer()` functions execute.

Note:

The driver's event triggers its function first and then the OI event triggers its function afterwards. The handler/prober driver's methods originate the events that call the driver or OI callback functions.

For the normal case where either the driver or OI does not have a function, the module that has the function executes its function. Place the callback functions in the OI instead of the driver so that the

same driver can be used on multiple systems that may or may not have the need for the functionality. Most Teradyne-supplied drivers do not include callback functions for this reason.

Methods

The most useful part of the EquipmentInterface is the methods. Most methods initiate events that call customizable functions in the driver code. Use this when making the driver responsive to various external events (such as End of Wafer or Out of Product) and allowing it to self-configure (Connect, Config, and so on).

The table lists methods for the EquipmentInterface control object.

Methods for EquipmentInterface Control Object

Name	Type	Arguments	Description
AssertEndOfCassette	N/A	(none)	Called by driver code to raise the EndOfCassette event.
AssertEndOfWafer	N/A	(none)	Called by driver code to raise the EndOfWafer event.
AssertOutOfProduct	N/A	(none)	Called by driver code to raise the OutOfProduct event.
AssertStartOfTest	N/A	(none)	Called by driver code to raise the StartOfTest event.
AssertStartOfWafer	N/A	(none)	Called by driver code to raise the StartOfWafer event.
Config	N/A	(none)	Shows the configuration form for the selected equipment driver. The driver must be disconnected to call this method.
Connect	Boolean	(none)	Initializes communication with the selected equipment driver.
DisableAllTesterSites	N/A	(none)	Tells the tester to stop testing all sites.
DisableTesterSite	N/A	Site As Integer	Tells the tester to stop testing the site.
Disconnect	Boolean	(none)	Terminates communication with the selected equipment driver.
EnableTesterSite	N/A	Site As Integer	Tells the tester to start testing the site.

ErrorConditionDetected	N/A	code As Integer mesg As String	Called by driver code to raise the ErrorDetected event.
GetWaferXY	N/A	Site As Long RetX As Long RetY As Long	For probers: reads wafer X and Y position for one site.
SendCommand	Boolean	Cmd As String	Writes a raw string command to the equipment
SendRequest	Boolean	Cmd As String, Reply As String Replylen As Integer	Writes a raw string command, and waits for a reply from the connected equipment.
SetWaferXY	N/A	Site As Long, X As Long Y As Long	For probers: sets wafer X and Y position for one site. In addition, programs wafer setup in DataCollection control, if any.
StartPolling	Boolean	(none)	Begins polling the connected equipment for test start signals.
StopPolling	Boolean	(none)	Breaks out of the polling loop.

Other than the Assert Methods, the most important methods are the DisableAllTesterSites, DisableTesterSite, and EnableTesterSite. These methods allow the driver to take the parsed test start information from the handler or prober and enable or disable sites that are to be tested. The Send methods allow for simple communication, although it is just as direct to make a call to the Communications object present that all have send and receive functions as well.

Note that use of any methods that have corresponding events normally calls a function in the driver or OI. As such, carefully consider inclusion of these methods in the driver code. Some functions are common or required (such as StartPolling and Connect) while others may or may not be present depending on the driver or OI structure (such as AssertOutOfProduct or AssertEndOfCassette).

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpabout.1.14

<	>
-----------------	-----------------

About Handler/Prober Communications : IG-XL Driver Elements : Communications Control

Communications Control

The communications control is what performs the physical communication with a handler or prober. Most of the driver code is written to support this control. The control creates a standard set of user functions that can be used, depending on which protocol is selected. Several communications protocols used on IG-XL-based testers, each of which has a unique set of properties, events, and methods. The protocols available are:

- | | |
|---|--------------------|
| • | Parallel Interface |
| • | RS-232 Interface |
| • | GPIB Interface |

See each section for information about each protocol’s communications control. If you want to call the communications control indirectly (for instance, using a multipurpose set of driver code), the EquipmentInterface offers methods that can call the respective communications functions. Usually, however, you should call the function from the Communications control to avoid confusion and improve clarity.

<	>
-----------------	-----------------

hpabout.1.15

<

>

About Handler/Prober Communications : IG-XL Driver Elements : Communications Timer

Communications Timer

The Timer control allows the driver to execute code at regular intervals by causing a Timer event to occur. The timer is a simple alarm clock or stopwatch. When regulating the communications poll cycle, however, it is invaluable. The timer is a standard Visual Basic object that is not part of IG-XL. The communications timer is what controls the polling cycle interval. This interval is a fixed time that the tester uses to space its polling cycles for information such as a test start. While a polling timer is not required, it is a convenient way to poll a handler or prober at a set interval. The alternative to using the timer is to have an infinite loop that continually polls the handler or prober. To avoid excessive or unnecessary polling, however, a timer that is set to a prudent time can maximize throughput while providing an interrupt-driven interface.

Properties

Properties for the timer allow you to enable or disable it as well as to set the interval between alarms. Typically, the interval is statically set at design time and the timer enabled or disabled as needed in the driver code. After the timer has triggered, disable the timer while processing the poll cycle so that the timer does not go off again and then enable it when the poll cycle is over. The table lists the driver-useful properties for the Visual Basic timer.

Properties for Timer Control Object

Name	Type	Description	Design Time	Run Time
Enabled	Boolean	Whether the timer is enabled to go off at the Interval or not.	R/W	R/W
Interval	Integer	Time in milliseconds that must elapse to trigger the Timer event.	R/W	R/W

Events

The timer has one purpose, to wait. When the interval wait expires, the timer triggers the Timer event that, in turn, calls the method.

Events for Timer Control Object

Name	Arguments	Description
Timer	(none)	Occurs when the Interval has elapsed calling the Timer method.

Methods

The timer’s sole method is the function where whatever actions should be taken at the interval are coded. In the context of a driver, the timer most often indicates the beginning of a poll cycle. The Timer method, therefore, typically includes a call to the polling function. In most handler/prober drivers, the polling function is PollForStart(). The name is unimportant, however, as long as it is called by the timer. It is important to disable the timer during the poll cycle to avoid having the timer expire during polling.

Methods for Timer Control Object

Name	Type	Arguments	Description
Timer	N/A	(none)	Called by driver code when the timer is enabled and the interval has elapsed.

hpabout.1.16

<	>
---	---

About Handler/Prober Communications : Handler/Prober Driver Flow

Handler/Prober Driver Flow

Each IG-XL handler/prober driver is independent and can have almost any functionality that you can implement in Visual Basic. Every IG-XL driver has a critical path flow enforced by IG-XL, however, in addition to some key components as previously discussed. This allows each driver to conform to a basic framework.

This figure shows the driver core flow.



IG-XL Handler/Prober Driver Core Flow

As the figure shows, the core flow that IG-XL enforces is small, giving authors freedom to design or modify a driver. In each of the calls to mandatory functions (such as Connect() and StartPolling()) code that performs setup for further processing is typically found such as setting variables, enabling other objects, and calling other optional functions and code. The GetControls(), initializeDefaultValues(), SetSimulatorMode(), Connect(), StartPolling(), StopPolling(), and Disconnect() functions are all required for the driver to be usable by IG-XL. The following table lists each of these functions. Other functions that may (and often do) exist in the driver are user configurable and optional and vary widely between drivers. Without the core functions, the driver cannot be loaded and used by IG-XL.

Required IG-XL Driver Functions

Function Name	Driver Location	Description, Notes, and Purpose
---------------	-----------------	---------------------------------

GetControls()	Class Object	Provides link to Driver to EquipmentInterface, allowing for information to pass to external modules
initializeDefaultValues()	Form Code	Public entry point into the driver, initializes any default values that will be used by the driver
SetSimulatorMode()	Form Code	Reads the value from the Sim button in the EquipmentSelect and sets variables inside river accordingly
Connect()	Form Code	Executes code that should happen once at the connection time, sets up communications interface for later use
StartPolling()	Form Code	Starts the polling process, typically using a Timer or an infinite loop
StopPolling()	Form Code	Aborts polling cycle, typically disabling Timer and pauses communications with Handler/Prober
Disconnect()	Form Code	Ends communication, disables further communication and unloads any driver-initiated processes

Note that EquipmentSelect actions are not physically linked in software, but the normal flow is shown using a dashed line. Most driver development occurs in the “Driver’s Test Cycle Executes” which is driven by the handler/prober communications interface selected as well as by the communications specification provided by the handler/prober manufacturer.

hpabout.1.17



About Handler/Prober Communications : Driver Installation and Maintenance

Driver Installation and Maintenance

Installing an IG-XL handler/prober driver is straightforward. If the driver is shipped with IG-XL, it is installed and registered when IG-XL is installed. No installation or system changes are required in this case. If the driver is a custom driver that was written but not shipped on the IG-XL CDROM, then the options are:

- If the source code is available and the tester has Visual Basic installed, then the driver can be compiled which will autoregister it.
- If the source code is not available or Visual Basic is not installed but the DLL is available, then the driver can be registered and configured to work on the tester using the regsvr32 command: `regsvr32 <Driver Name and Path>`

To uninstall a driver, call regsvr32 with the "/u" option preceding the driver DLL name:

`regsvr32 /u <Driver Name and Path>`

While it is not required to uninstall a driver first when updating it with a new version, you should do so to avoid any conflicts. Once a driver is uninstalled, it no longer appears in the EquipmentSelect menu. However, it is not deleted from the tester. For reliability reasons, delete any drivers not in use after uninstalling them.

Note that when a driver is installed, the Windows registry keeps track of the driver so there is no requirement on the location of the driver. In most cases, you should keep drivers in a central location. All IG-XL drivers created by Teradyne are found in the IG-XL \bin directory for each IG-XL version installed. The suggested practice for maintaining custom drivers is to place them in a separate directory in the main IG-XL directory. This prevents accidental loss of the driver if the IG-

XL directory is deleted, separates stock drivers from custom drivers, and allows multiple versions of IG-XL to share a driver.

< >	
	2015 Teradyne, Inc. All rights reserved.

hpfuncts.7.1

<	>
-----------------	-----------------

Handler/Prober Function Library

Handler/Prober Function Library

This section describes the functions from various IG-XL components that are commonly used in IG-XL handler/prober drivers. Note that to use the functions, the respective IG-XL object must be on the OI (TesterControl and EquipmentSelect) or driver (EquipmentInterface, NI32HSControl, RS232Control, and GPIB Control). If none of the functionality is required, then you need not include the object on the form. For more information on each object, see additional sections in IG-XL Help.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpfuncts.7.2

<

>

Handler/Prober Function Library : NI32HSControl Functions

NI32HSControl Functions

Function	ConfigurePort
Synopsis	Public Function CreatePort(BoardID, PortType, LSB, MSB, Encoding, SignalStart, SignalLength, OutOfRangeVal) As Integer
Description	This function configures a port based on the LSB/MSB, received as line numbers, and from other characteristics. The range of lines is checked against other ports to verify that there is no overlap with other ports. If there is overlap, then an error is flagged.
Parameters	BoardID: Integer BoardID assigned by the NI32HS Configuration utility (1, 2, 3, etc.) PortType: Enumerated Either Input or Output LSB: Integer Line number corresponding to LSB, valid range is 0 – 31 MSB: Integer Line number corresponding to MSB, valid range is 0 – 31 Encoding: Enumerated Either binary or bit-shift SignalStart: Long The number of milliseconds to delay signal assertion. Valid for output only. Maximum value 10000. SignalLength: Long The number of milliseconds to assert a signal (for output signals). Maximum value 10000 or the number of milliseconds a signal needs to be asserted to be considered valid (for input signals).

OutOfRangeVal: Integer Value that is written to the port if you attempt to write a value that can't be encoded in the number of bits (lines).

Return Value

-1 on error otherwise a PortID (integer value). The Port ID associates an integer value with a series of lines on a port on a particular board. Ports spread out across multiple boards are not permitted.

Function

ReadPortConfig

Synopsis

Public Function ReadPortConfig(portID) As Boolean

Description

Adds this port/line to the input mask for input operations. When the input operation begins, if any lines (for a multiline input port) are triggered for the specified time the operation completes and the value is returned. Calls an internal function to configure the port group that PortID belongs to for an Input operation.

Parameters

PortID: Integer PortID assigned to this port by a previous call to ConfigurePort.

Return Value

True if the PortID is valid otherwise False

Function

WritePortConfig

Synopsis

Public Function WritePortConfig(PortID, DataValue) As Boolean

Description

Writes the value to the output array, to the bits corresponding to the lines that comprise the port. The data is appropriately encoded as it is written.

If the data value cannot be written in the number of bits available, then the specified out of range value is written. If no out of range value has been specified, then an error is returned.

The value is written to an array of either longs of integers (depending on the number of ports used). The board is programmed so that each element represents assertion of the value for 1ms. Zeros (signal not asserted) are written to the output array for the number of milliseconds specified in SignalStart. Ones are then written for the number of milliseconds corresponding to SignalLength.

Parameters

PortID: Integer Port ID returned by PortConfigure

DataRow: Integer The data to write to output.

Return Value True if the data value (or out of range value) could be written otherwise False.

Function InitiateWrite

Synopsis Public Function InitiateWrite() As Boolean

Description For multiboard systems, slave boards are set up to write when the master board write operation begins. Data buffers are passed to the boards, and the write operation is initiated on the master board. For single board systems, the data is written to the board, and the write operation Immediately starts. If handshaking is enabled, then an input operation is initiated concurrently. To get wait for the handshake line, the driver waits until the handshake line is asserted or until the handshake time out expires. If the handshake is not received within the specified time, an error is flagged. When the output operation completes, the output array is cleared while the handler/prober is indexing.

Parameters None

Return Value True if the setup and the write succeeded otherwise False.

Function InitiateRead

Synopsis Public Function InitiateRead() As Boolean

Description For multiboard systems, the slave boards are set up to write when the master board's read operation is initiated. Cleared data buffers are passed to the boards. These buffers are filled when any input line is triggered for the minimum value input signal time. In single-board systems, an empty buffer is passed to the board and the read operation is started.

Parameters None.

Return Value True if the read operation could be successfully initiated, otherwise False.

Function	InputReady
Synopsis	Public Property InputReady() As Boolean (Read Only).
Description	This property is set to True automatically whenever any input line has been asserted
Parameters	None
Return Value	True if any input line has been asserted long enough to be considered valid, otherwise False.

Function	HandshakingEnabled
Synopsis	Public Property HandshakingEnabled() As Boolean (Read/Write)
Description	This property allows you to enable or disable handshaking and to query its state. If handshaking is enabled, a read operation is started at the same time as a write operation. If the handshake signal is not received within the specified length of time, an error condition is flagged.
Parameters	True/False
Return Value	True/False

Function	ResetBoard
Synopsis	Public Function ResetBoard(boardNumber) As Boolean
Description	This function allows you to reset a board.
Parameters	boardNumber: Integer Board number assigned during installation.
Return Value	If reset was successful True, otherwise False

<

>

hpgpib.5.1



GPIB Interface

GPIB Interface

The General Purpose Interface Bus (GPIB) interface offers significantly more flexibility than parallel or serial interfaces. At the same time, the GPIB interface also can be more complex to design and debug. Most handlers and all probers support a heavily developed GPIB as a standard option. If a GPIB interface can be used, therefore, the full abilities of the communications interface can be effectively used.

The GPIB is a communications bus that was developed by Hewlett-Packard to interconnect electrical engineering lab equipment. Most oscilloscopes, wave generators, logic analyzers, spectrum analyzers, and bench multimeters support GPIB. In general, most test and measuring equipment made by HP and other manufactures uses GPIB to transmit sampled data and receive configuration instructions to a host or other compatible equipment.

Teradyne testers support GPIB, which is defined by IEEE standards 488.1 and 488.2. GPIB, also known as the IEEE 488 interface and sometimes just the IEEE interface, has standard hardware (connectors and cables) and standard electrical properties (wiring, signal levels, and signal usage). Unlike RDP (see RS-232 Interface), GPIB (IEEE 488.1) does not have a standard messaging command set.

GPIB consists of an 8-bit data bus with five control lines. In addition, three handshaking lines ensure that any data sent is received. The handshaking and control protocol is similar to RDP; however, GPIB uses TTL signals on a parallel bus so information can be transmitted much faster. You need not understand the bus control and handshaking for normal operation since the GPIB hardware and software in the tester maintains the bus. In the event that some debug is required on a GPIB interface, however, the transitions and states of the handshaking and control lines can be critical. Several commercially available texts provide in-depth analysis of IEEE 488 and its properties. The best all-purpose reference is IEEE-488 General Purpose Instrumentation Bus Manual (1989) by Anthony J. Caristi (ISBN 0-12-159820-9).

GPIB has many advantages. It offers a common interface that buses multiple devices. Any IEEE 488 cable works with any IEEE 488 device as long as the total length of the cable is not longer than 2 meters times the number of devices attached. Since the IEEE 488 standards define most of the mechanical and electrical properties of GPIB, no wiring is required as on a parallel interface. The data rate of GPIB is device-dependent (a streaming protocol) but can range as high as 1 Mbps. The GPIB data rate pushes the comparative minimum theoretical communications time to less than 1 ms per device. GPIB also allows advanced messaging using single bytes, or two hex numbers, to transmit information over the interface instead of text commands.

The main disadvantage of GPIB is that the IEEE 488 standard does not define the messaging between two devices on the bus. IEEE standard 488.2 defines a standardized messaging scheme for communicating between devices on the GPIB IEEE 488.2 was historically never used by ATE, however, so it is not supported on the tester. The messaging scheme is created by the manufacturer of the equipment, in this case the manufacturer of the handler or prober. Most interfaces vary with little similarity between manufacturers and in some rare cases, even models made by the same manufacturer. This lack of conformance means that setting up GPIB software on a tester takes slightly longer than other communications interfaces. In addition, if there are problems with a GPIB interface, a GPIB Analyzer can be required to debug the problem.

The following topics are available:

- Hardware
- Software
- IG-XL Driver
- Important Notes

hpgpib.5.2

<	>
---	---

[GPIB Interface](#) : Hardware

Hardware

IG-XL-based testers use the National Instruments GPIB-PCI model GPIB hardware that is fully IEEE 488 compliant. The GPIB card is installed in an open PCI slot in the tester’s computer. Standard IEEE 488 cables are use throughout and an IEEE 488 port is provided on the tester’s bulkhead panel for handler/prober interfacing and external instruments.

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpgpib.5.3

<	>
-----------------	-----------------

[GPIB Interface](#) : Software

Software

IG-XL-based testers use the standard National Instruments (NI) GPIB driver for Windows NT/2000. All configuration is made in the IG-XL driver. All IG-XL-based testers come with the GPIB hardware and software preinstalled so no installation of any software is required unless the NI driver needs updating. For information on installing the NI GPIB software, consult the IG-XL Software Installation Guide.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpgpib.5.4

<

>

GPIB Interface : IG-XL Driver

IG-XL Driver

Unlike the ParallelInterface and P849 drivers on the tester, each handler or prober has a separate driver. This is due to the custom configuration that is required by each handler/prober manufacturer. However, IG-XL provides a standard set of controls and functions that can be used for communication. In addition, several common drivers are shipped with IG-XL for which the source code is available in the IG-XL \src directory.

Prewritten and installed drivers are as follows:

Teradyne-Supplied IG-XL GPIB Drivers

Manufacturer	Type	Model(s)	Notes
Accretech	Prober	All	Formerly TSK
Advantest	Handler	4182	
Aetrium	Handler	5050QTS 5800	
ASA	Handler	All	
ASECO	Handler	All Others S130	
Daymarc	Handler	3287	
Delta	Handler	Castle Edge Flex	

		RFS	
Electroglas	Prober	All	
KLA	Prober	All	Now TEL (Probers Only)
MCT	Handler	Tapestry	
Multitest	Handler	All	9918 has separate driver
Rasco	Handler	SO1000	
Seiko-Epson	Handler	All	
	Prober	All	
TEL	Prober	All	

Note that Teradyne periodically updates, adds, and improves GPIB drivers that ship with new versions of IG-XL. Therefore, save any changes to the existing code separately to ensure they are not lost in an IG-XL update. The list in the table is not exhaustive and is not a suggested list of handler/prober vendors.

<

>

2015 Teradyne, Inc. All rights reserved.

hpgpib.5.5

<	>
-----------------	-----------------

[GPIB Interface : IG-XL Driver : Configuration](#)

Configuration

The configuration of a GPIB driver varies from driver to driver. Like all other IG-XL handler/prober drivers, however, a configuration form is the norm. Depending on the handler or prober manufacturer's GPIB implementation, the configuration form may be simple (selecting only the GPIB address, terminator, and number of sites) or it may be more complex (including site mapping, compatibility options, and so on). The requirements of the configuration form should match the driver's functionality so that each item on the form matches a function or requirement inside the driver. For an example, see [Configure Form for Electroglas GPIB Interface](#).

<	>
-----------------	-----------------

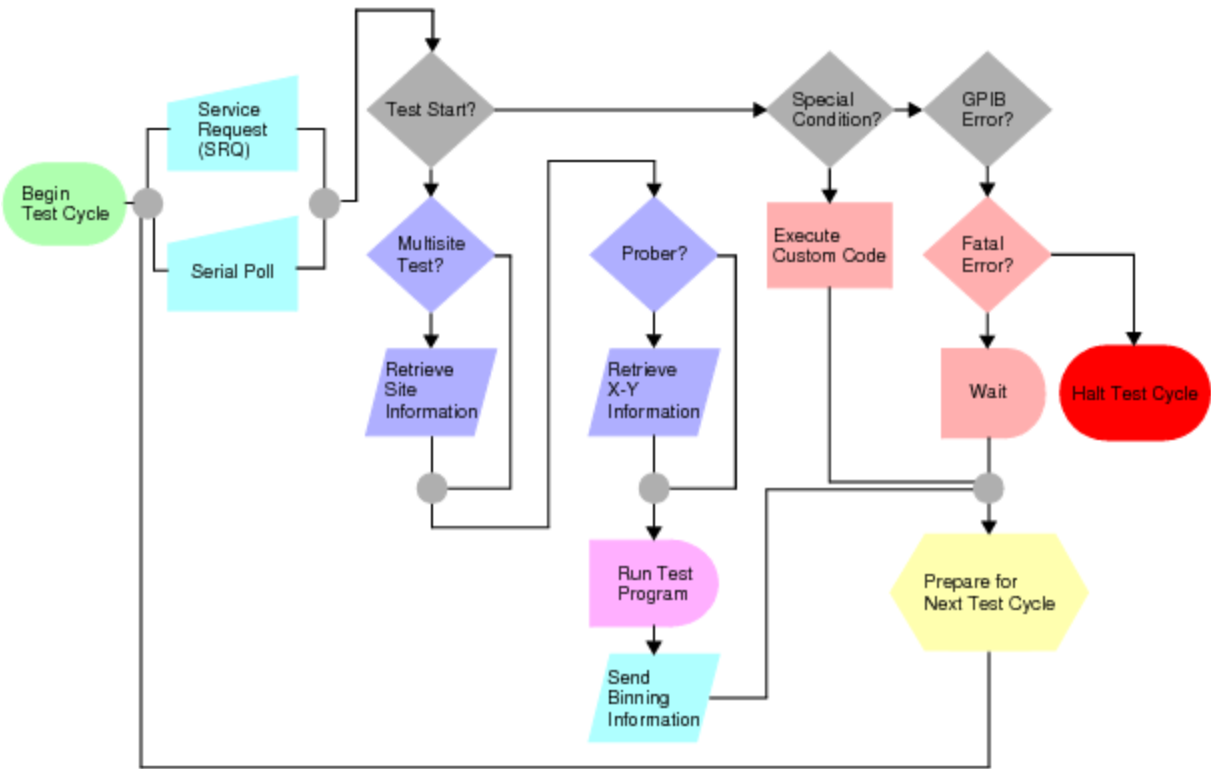
	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpgplib.5.6

GPIB Interface : IG-XL Driver : Flow

Flow

The test cycle flow for a typical GPIB driver is shown in the figure. Note that every manufacturer has a specific implementation, so reference that manufacturer’s documentation for details. However, all GPIB communication cycle flows will look similar: test start, retrieve coordinate/site information, and binning. For the exact implementation, see each manufacturer’s GPIB interface documentation.



GPIB Driver (Typical) Test Cycle Flow

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpgpib.5.7

<

>

GPIB Interface : IG-XL Driver : Organization

Organization

The unique part of each GPIB interface is the GPIBControl object that is found on each GPIB driver form. Each GPIBControl object has a standard set of Visual Basic properties, events, and methods. For an example, see the form in IG-XL Driver Elements.

Properties

The properties for the GPIBControl object are listed in the table. The Read/Write status of the property is listed in the two rightmost columns. Properties that are read only can only be read by the driver, whereas the properties that are both Read and Write can be altered.

Properties for the GPIBControl Object

Name	Type	Description	Design Time	Run Time
AssertEOI	Boolean	A GPIB signal used to terminate bus transfers. If True, the driver asserts EOI to terminate write operations and terminate read operations when the equipment has asserted the EOI signal.	N/A	R/W
eosChar	Byte	Character used to terminate bus transfers (Read/Write). Either carriage return or line feed.	N/A	R/W
eosMode	Integer	A bit field that specifies how the End of Sequence (EOS) character is used. The EOS character is the last character of the read/write terminator. If 0800H is set, the	N/A	R/W

		GPIB driver asserts EOI when the EOS is written. If 0400H is set, read operations are terminated by the EOS character, if 1000H is set all 8 characters of the EOS are compared.		
gpibAddr	Integer	The GPIB address of the equipment (1 – 30)	N/A	R/W
gpibIBErr	Integer	Value corresponds to the GPIB error word (iberr)	N/A	R
gpibIBSta	Integer	Value corresponds to the GPIB status word (ibsta)	N/A	R
ReadRetryAttempts	Integer	Number of times a read operation should be retried, if an error occurs during the initial operation.	N/A	R/W
readTerminator	String	The character or characters that will be used to terminate read operations, typically carriage return, line feed or a combination of carriage return and line feed. Some equipment specifies different terminators for reads and writes.	N/A	R/W
SimulatorMode	Boolean	Allows the driver to simulate the GPIB. It communicates with the equipment simulator rather than using the GPIB hardware to R/W.	N/A	R/W
SpollBeforeRead	Boolean	Instructs the GPIB to wait for the equipment to assert a service request (SRQ), to indicate it is ready to transmit data, before the driver performs a read.	N/A	R/W
SpollRetryAttempts	Integer	Number of times an Spoll operation should be retried, if an error occurs during the initial operation.	N/A	R/W
timeout	Integer	Number of milliseconds to wait for bus operations (Serial Poll, Read or Write).	N/A	R/W
WriteRetryAttempts	Integer	Number of times a write operation should be retried, if an error occurs	N/A	R/W

		during the initial operation.		
writeTerminator	String	The character or characters that are used to terminate write operations, typically carriage return, line feed or a combination of carriage return and line feed. Some equipment specifies different terminators for reads and writes.	N/A	R/W

Events

This table identifies the Visual Basic events that the GPIBControl can detect. Handler/prober events (such as End of Wafer and End of Lot) are detected by the EquipmentSelect control object in the Operator Interface or by the EquipmentInterface control object on the driver form. In the case of the GPIBControl object, all the events are simulator events and are, therefore, not typically used during production testing.

Events for the GPIBControl Object

Name	Arguments	Description
gpibInitializeEvent	None	Raised if the SimulatorMode property is True. Used to indicate to the equipment simulator that it should initialize.
gpibReadEvent	buf as String buflen as String	Raised if the SimulatorMode property is True. Indicates to the equipment simulator a read operation is occurring. The "reply" argument contains the reply from the simulator. The replyLen value is passed to the simulator. This is the number of characters that is allocated for "reply"; its use in the simulator is optional. Returns the number of characters read, excluding the terminator.
gpibSerialPollEvent	sbyte as Integer	Raised if the SimulatorMode property is True. Instructs the equipment simulator to the GPIB control is performing a serial poll. The simulator sets Sbyte to the value of the status byte.
gpibWriteEvent	buf as String buflen as String	Raised if the SimulatorMode property is True. Sends the equipment simulator the string specified in cmd, with writeTerminator appended, and the length of

cmd. Returns the number of characters written excluding the terminator.

Methods

This table lists the methods that are available for use with the GPIBControl object. These functions perform the communication with the handler or prober so no direct calls to NI GPIB functions are required.

Methods for the GPIBControl Object

Name	Type	Arguments	Description
gpibDisconnect	Sub	None	Sends the equipment a command to put it off line.
gpibInitialize	Integer	None	Initializes the communication interface with the equipment using the GPIB properties and GPIB Address. In simulator mode, it calls the simulator initialization function.
gpibPerformSpoll	Boolean	sbyte as Integer	Performs a serial poll. Returns True if the equipment was requesting service; otherwise; returns False. If True, the argument contains the value of the Status Byte, returned by the Serial Poll.
gpibRead	Integer	readbuf as String buflen as Long	Allocates memory for reply and reads the equipment response from the bus and returns the number of characters read, excluding the terminator. The second argument, replyLen, specifies the number of bytes to allocate.

gpiSendRequest	Integer	req as String reply as String replyLen as Integer	Writes the command in cmd, followed by the writeTerminator to the bus. If the write was successful, memory is allocated for reply (specified by replyLen) and waits for the equipment to send a reply. If the read operation was successful, the number of characters read (excluding the readTerminator) is returned. Returns -1 if read/write error.
gpiWaitForSRQ	Boolean	none	Waits for the equipment to assert a service request or for the time-out period. If a service request was asserted, returns True, and sbyte contains the value of the Status byte. If no SRQ asserted, then returns False, and the value of sbyte is undefined.
gpiWrite	Integer	buf as String buflen as Integer	Writes the string in 'cmd' to the bus, followed by the writeTerminator. If successful, the number of characters written is returned; otherwise, -1 is returned.

hpgpib.5.8

<

>

GPIB Interface : IG-XL Driver : Functions

Functions

The GPIBControl object functions listed in the following table are used to access the GPIB hardware in the tester. Each function can be used at any time in the driver to communicate with the handler or prober. Since they are part of the GPIBControl object, they can be used by any driver that uses the GPIBControl on its form.

Functions for the GPIBControl Object

Name	Type	Arguments	Description
gpibChangeAddr	Integer	newAddr as Integer	Changes the primary address of a GPIB device, once communication has been initialized
gpibChangeAssertEOI	Boolean	newEOI as Boolean	Can be used to change whether EOI is asserted once the interface has been initialized
gpibChangeTimeout	Boolean	newTimeout as Integer	Changes the time-out (in milliseconds), after communication has been initialized
gpibDefaultTimeout	Integer	(none)	Returns the value of the default time-out value
gpibPerformSPoll	Boolean	sbyte as Integer	Perform a serial poll on the device

gpibSendSelectedDeviceClear	Integer	(none)	Send SDC (selected driver clear) message to the device
SetAssertEOIwithEOSonWrite	(none)	assert as Boolean	Determines whether the driver will assert the EOI signal when it writes the EOS character
SetEightBitCompare	(none)	assert as Boolean	If true, all 8 bits of the EOS character are compared to determine its presence. Otherwise only 7 characters are used
SetEOSChar	(none)	eosCh as Byte	Sets the EOS character
SetGPIBTimeout	(none)	TimeoutMS	Sets the Timeout value (in milliseconds) for GPIB bus operations
SetTermReadOnEOS	(none)	assert as Boolean	If true, read operations are ended when the EOS character is encountered

hpgpib.5.9

<	>
-----------------	-----------------

GPIB Interface : Important Notes

Important Notes

When writing a GPIB driver, it is often easiest to take an existing driver that is close or similar and modify it instead of creating a completely new driver from a blank form. Most handler/prober GPIB interfaces are similar, so most common “templates” are available with the tester (see the list of drivers in IG-XL Driver).

To copy an existing driver, copy all original driver files and place them in a new separate directory to avoid altering the standard driver. When modifying an existing driver, keep a backup in a separate location (for example, the main IG-XL directory) to avoid losing any driver changes due to a future IG-XL software update.

To debug the GPIB interface, the tester comes with the National Instruments NISpy utility. NISpy is used to monitor the GPIB communications at the bus level. While NISpy does not give in-depth information, it is useful for debugging common problems. If you need a more in-depth problem-solving tool, you can purchase the National Instruments GPIB Analyzer directly from National Instruments.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpp849.8.01

<	>
-----------------	-----------------

P849 Communications Interface

P849 Communications Interface

This section describes the P849 communications protocol. The protocol follows ISO standards and allows functional commands and status transactions to be placed under a fully automatic production testing process.

The following topics are available:

- | | |
|---|------------------------------------|
| • | Physical Layer |
| • | Datalink Layer |
| • | Network Layer |
| • | Transport Layer |
| • | Session Layer |
| • | Presentation Layer |
| • | Applications Layer |
| • | Reference Section |

Purpose

To define a fast and intelligent working control interface between testers and handlers or probers. Throughout this specification, handlers and probers will be referred to as peripherals.

Scope

This specification defines the minimum necessary levels of interface to allow test systems and peripherals to communicate all functional commands and status transactions for support of a fully automatic production testing process.

Format

The protocol follows the ISO Standard seven layers: Physical, Data Link, Network, Transport, Session, Presentation, and Application. These are described as follows.

Physical	This layer provides a physical connection for the medium between the test system and the peripheral.
Data Link	This layer acknowledges the messages exchanged between equipment on a byte by byte basis.
Network	This layer is not used in this application.
Transport	This layer assembles the messages received from the equipment.
Session	This layer manages the dialogue among multiple pieces of equipment that includes more than two talkers/listeners.
Presentation	This layer is not used in this application.
Application	This layer defines the commands and status requests that are communicated between the testers and handlers or probers.

hpp849.8.02

<

>

P849 Communications Interface : Physical Layer

Physical Layer

Implementation of the physical layer is using an RS232C serial link. The RS232C connector is comprised of the following pins on a 25 pin D connector:

Pin 1	Shield ground
Pin 2	Data out
Pin 3	Data in
Pin 7	Signal Ground

The female connector will be mounted on the peripheral. A male connector will be mounted on the tester. The cable is wired as follows:

Cable Wiring

Male/Peripheral	Female/Tester
Pin 1	Pin 1
Pin 2	Pin 3
Pin 3	Pin 2
Pin 7	Pin 7

<	>	
	2015 Teradyne, Inc. All rights reserved.	

hpp849.8.03

<	>
-----------------	-----------------

P849 Communications Interface : Datalink Layer

Datalink Layer

The following information describes the Datalink Layer requirements for the P849 Communications Interface.

- | |
|--|
| <ul style="list-style-type: none">• RS232C |
|--|

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpp849.8.04

<

>

P849 Communications Interface : Datalink Layer : RS232C

RS232C

Baud Rate	Default requirement of 9600. Other baud rates optional (300, 1200, 2400, 4800, 19200, 38400).
Start/Stop Bit	One start bit and one stop bit
Character Length	Eight (8) bits
Parity	None
Character Buffering	Each end of the communication link must be able to store 1 character until that character can be read.

The use of a single character start of text and single character end of text allows variable length strings to be sent as data. These strings contain a single character transaction type code and a single character command followed by any argument.

Legal Protocol Character Set

The following characters are used in the protocol:

Legal protocol character set for the protocol is: ENQ, ACK, NAK, STX, ETX, or a member of letter, digit, or special ASCII code.

ENQ	(ASCII character 5) will be used to request permission to send a command and/or response string.
ACK	(ASCII character 6) will acknowledge that the tester or peripheral is ready to accept the command and/or response string. It is the obligation of

	the receiving station to read all data between STX (start of text) and ETX (end of text).
NAK	(ASCII 25) - will be used by the peripheral to indicate that it is too busy to respond at the current time. It is an alternate means to avoid time-out situations.
STX	(ASCII 2) signifies the begin of text. Beginning block code delimiter.
ETX	(ASCII 3) signifies the end of text. Note that ETX is compatible with many data networking systems. End block code delimiter.
NULL	Is the sequence STX ETX with no text between.

The message and response strings may be made up of either the NULL sequence or any ASCII character in the range hex 20 through hex 7F.

Protocol Contention Handling

During all communication sequences, contention will be resolved by having the tester serve as the controller and the peripheral as the slave. The peripheral can only respond to requests for information. It cannot send unsolicited data to the tester. Each end of the communication link must be able to re-sync on the ENQ-ACK sequence at any time. This is necessary to handle possible time out problems. The ENQ-ACK sequence without a command string can also be used to determine if a peripheral is present on the node.

A time out condition occurs if there is greater than 1 second between:

- Issuance of ENQ and receipt of ACK or NAK
- Any 2 characters in a command or response string

If a time out occurs, the entire ENQ-ACK sequence and the command or response string that coincides must be repeated. If there are more than three retries, it will be considered a catastrophic error.

<	>
	2015 Teradyne, Inc. All rights reserved.

hpp849.8.05

<	>
-----------------	-----------------

P849 Communications Interface : Network Layer

Network Layer
Not Applicable

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpp849.8.06

<	>
-----------------	-----------------

P849 Communications Interface : Transport Layer

Transport Layer

The following information describes the Transport Layer requirements for the P849 Communications Interface.

- | | |
|---|---|
| • | Protocol Syntax |
| • | Syntax for Message From Tester to Peripheral |
| • | Syntax for Response String Returned From Peripheral to Tester |
| • | Nonimplemented Commands |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpp849.8.07

<	>
-----------------	-----------------

P849 Communications Interface : [Transport Layer](#) : Protocol Syntax

- Protocol Syntax
- Single character transaction type code.
- Single character command.
- Variable length error messages and number strings.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpp849.8.08

<

>

P849 Communications Interface : Transport Layer : Syntax for Message From Tester to Peripheral

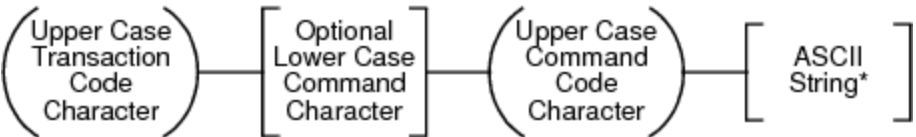
Syntax for Message From Tester to Peripheral

A message comprises command characters and string (or numeric) text:

Tester to Peripheral Message Syntax

Command	Communication Flow
ENQ	Tester to peripheral
ACK	Peripheral to tester
STX	Tester to peripheral
<message>	Tester to peripheral
ETX	Tester to peripheral

where <message> is of the form:



*ASCII strings are defined in the applications layer (Section 8.0).

Format of P849 Messages

<

>

hpp849.8.09

<

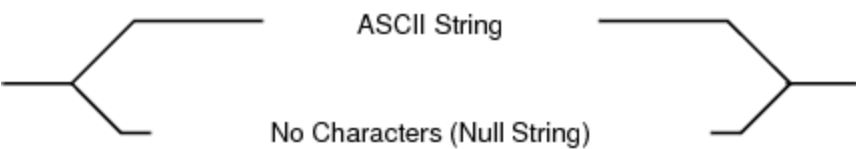
>

P849 Communications Interface : Transport Layer : Syntax for Response String Returned From Peripheral to Tester

Syntax for Response String Returned From Peripheral to Tester

Peripheral to Tester Response String Syntax

Command	Communication Flow
ENQ	Peripheral to tester
ACK	Tester to peripheral
STX	Peripheral to tester
<response string>	Peripheral to tester
ETX	Peripheral to tester



*ASCII strings are defined in the applications layer (Section D8.0).

Format of P849 Responses

<

>

hpp849.8.10

<	>
-----------------	-----------------

P849 Communications Interface : [Transport Layer](#) : Nonimplemented Commands

Nonimplemented Commands

A peripheral must be prepared to accept all commands or requests even if the function is not supported. Nonsupported commands must be ignored. Nonsupported requests must respond with a null string.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpp849.8.11

<	>
-----------------	-----------------

P849 Communications Interface : Session Layer

Session Layer
Not Applicable

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpp849.8.12

<	>
-----------------	-----------------

P849 Communications Interface : Presentation Layer

Presentation Layer
Not Applicable

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpp849.8.13

<

>

P849 Communications Interface : Applications Layer

Applications Layer

The application commands described in this specification are the minimum set of commands necessary to implement a basic communications package between a tester and a peripheral. Two examples of this are shown in [Sample Probing Flow](#) and [Sample Handler Flow](#). New applications-specific commands can be generated, provided that they follow the definitions in layers 1 though 6 and are cleared by the P849 working group to avoid conflicts.

Four types of application transactions are required between the test system and a peripheral.

Action Commands	Consists of data to initialize the peripheral before testing may begin. Also includes commands requesting the peripheral to take specific action (for example, motion).
Categorization Commands (Bin information)	Consists of binning or category information on the current device or devices under test.
Status Commands	These are requests for information that the tester makes to the peripheral. The peripheral must respond with a reply or a null message within the time-out limit.
Bulk Data Transfer Commands	These are commands for receiving and transmitting bulk data in the form of blocks containing records of up to 256 characters in length.

The application commands for each of these categories are listed in the following sections. Syntax definitions used in the command arguments for the commands include:
These are commands for receiving and transmitting bulk data in the form of blocks containing records of up to 256 characters in length.

n	Number, + or -
x,y	Number, coordinates
c	ASCII single character
s	String, max 256 characters in length
b	Boolean value (0,1), (ASCII characters)
{ }	Optional argument
[]	Repeated optional arguments

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hpp849.8.14

<

>

P849 Communications Interface : Applications Layer : Action Commands

Action Commands

AcL

Name	Probe 1 wafer
Applicable To	Probers
Syntax	AcL
Response	None
Description	This command assumes that the first die is under the probes. When the command is received, the prober begins probing the wafer on the chuck. When probing is complete, the wafer will not be in contact with the probes, the “End of Wafer” flag will be true, and the “End of Product” flag will be valid.
Example	None

AcM

Name	Abort Probing
Applicable To	Probers
Syntax	AcM
Response	None
Description	This command stops probing. The Z stage is lowered to the out of contact point and the XY motor is left wherever it is. The “End of Wafer”

flag will be set, and the "End of Product" flag will be valid.

Example

None

AcS

Name Handle wafer

Applicable To Probers

Syntax AcS

Response None

Description When the command is executed, the prober:

1. Unloads the wafer on the chuck.
2. Loads and aligns a new wafer.
3. Moves the "first" die under the probes.

After step 3, the Z stage is left with the wafer out of contact.

Example

None

AdB

Name Move Z up

Applicable To Probers

Syntax AdB

Response None

Description If the Z stage is down, it moves up according to the current Z travel mode. If the Z stage is already up, no Z motion takes place. "Up" means into contact with the probes.

Example

None

AEs

Name Application specific commands

Applicable To Handlers

Syntax AEs

Response	None
Description	This command is the application specific command for handlers. See Manufacturers Manual for string definitions.
Example	None

AeK

Name	Assign good die bins
Applicable To	Probers
Syntax	AeKb[,b]
Response	None
Description	This command allows the host to define the “good” bin codes and the bin codes that are “bad” die. This has nothing to do with inking! “Good” and “bad” are used for accumulating die total counts. The first b corresponds to bin 0, the next to bin 1, etc. Setting a b to 1 means that bin is a good die, b = 0 means that it is a bad die. The number of bins is variable in length up to 31.
Example	To assign bins 0,5,6,7,8 and 9 as “bad” bins: AeK0,1,1,1,1,0,0,0,0,0

AfC

Name	Assign logical ink codes
Applicable To	Probers
Syntax	AfCn[,n]
Response	None
Description	This command defines (for single site probe only) what inkers are fired in response to any given incoming bin code. The first n corresponds to bin 0, the next n to bin 1, etc. The inkers defined for n are listed in the following table. There may be up to 32 n’s in the string to give one per bin.

Bins and Inkers

n	Inkers			n	Inkers			
0				8				
1	1			9	1			4
2		2		10		2		4
3	1	2		11	1	2		4
4			3	12			3	4
5	1		3	13	1		3	4
6		2	3	14		2	3	4
7	1	2	3	15	1	2	3	4

Example

To define bin 0 to fire inker 1, bin 15 to fire inker 2 and all other bins to fire all four inkers:
AfC1,15,15,15,15,15,15,15,15,15,15,15,15,15,15,2

AgM

Name	Clear alarm
Applicable To	Probers and Handlers
Syntax	AgM
Response	None
Description	This command turns off the audible or visual alarm. If enabled, the alarm stops. Note that no problems are fixed by this command, corrective action must still be taken by the operator or tester.
Example	None

AKs

Name	Load setup file
Applicable To	Probers and Handlers

Syntax	AKs
Response	None
Description	This command requests the prober or handler to load all of its own setup parameters from a file named <i>s</i> located within its own memory. <i>s</i> is an ASCII string of variable length up to 256 characters.
Example	To load the file named 7400 setup: AK7400 setup

AdA

Name	Move absolute die
Applicable To	Probers
Syntax	AdAx,y
Response	None
Description	The motor moves to die coordinate x,y. The motor moves only in whole die steps, it will not move a part of a die. The Z stage is lowered before X,Y travel and then raised again.
Example	None

AdC

Name	Move Z down
Applicable To	Probers
Syntax	AdC
Response	Z motion
Description	If the Z stage is up, it moves down according to the current Z travel mode. If the Z stage is down, no Z motion takes place.
Example	None

AdI

Name	Move relative die
------	-------------------

Applicable To	Probers
Syntax	Adlx,y
Response	None
Description	This command causes the prober to step x die positions in the X axis and y positions in Y. If the Z stage is up before motion, it is lowered before moving and raised again when XY travel is complete.
Example	None

AdJ

Name	Move relative machine
Applicable To	Probers
Syntax	AdJx,y
Response	None
Description	The motor moves x 0.1 mil steps in the X axis and y 0.1 mil steps in Y. Refer to specific prober manuals for definition of coordinate directions.
Example	None

AdK

Name	Move Z to specific height
Applicable To	Probers
Syntax	AdKn
Response	None
Description	The Z stage moves to the height specified by n. After this command is successfully executed, the prober considers the Z stage to be in the "up" position, no matter the actual height.
Example	None

ALs

Name	Applications specific commands
------	--------------------------------

Applicable To	Probers
Syntax	ALs
Response	None
Description	This command is the application specific command for probers. See specific Manufacturer's Manuals for string definitions.
Example	None

AdE

Name	Unload wafer
Applicable To	Probers
Syntax	AdE
Response	None
Description	This command unloads the wafer from the chuck without loading the next wafer.
Example	None

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hpp849.8.15

<

>

P849 Communications Interface : Applications Layer : Categorization Commands

Categorization Commands

BA

Name	Bin to category/ink
Applicable To	Probers and Handlers
Syntax	BA n_1,n_2,n_3 [: n_1,n_2,n_3]
Response	None
Description	This command causes the device to be binned according to bin n_3. Upon receipt of this command, the peripheral indexes to the next device under test and the "Start Test" flag becomes true when the next device is ready in contact. n_1 is the argument used to specify the site number for parallel testing. The default is "1" for single site testing. On probers, n_2 is ignored for single site probing, but for multisite or parallel probe, n_2 identifies the inker number to fire. (For single site probing the ink commands are derived from n_3 and the logical ink codes assigned. n_3 is the argument that defines the bin number for the device tested. This must be an ASCII integer in the range 0 through 31. For parallel sites, the tester only responds with binning corresponding to those sites that received a "start test" signal.
Example	To bin a single site DUT to bin 3: BA1,0,3

To bin a dual probe configuration with site #1 as bin 2 with no ink and site #2 as bin 11 with inker #2 fired:
BA1,0,2;2,2,11

BB

Name	Break device contact
Applicable To	Handlers
Syntax	BBn[,n]
Response	None
Description	This command causes the contactors at the specified sites (n) to be opened and electrical contact to be broken. Other sites may be opened and closed in the process.
Example	To open and break contact on sites 1,7 and 8: BB1,7,8

BC

Name	Make device contact
Applicable To	Handlers
Syntax	BCn[,n]
Response	None
Description	This command causes the contactors at the specified sites (n) to be closed. Other sites may be opened and closed in the process.
Example	To close sites 2,5, and 7: BC2,5,7

hpp849.8.16

<

>

P849 Communications Interface : Applications Layer : Status Commands

Status Commands
CA

Name	Request error code/inhibits
Applicable To	Probers and Handlers
Syntax	CA
Response	Query
Description	The peripheral responds with a single character error code followed by the ASCII text of the error message. Any response other than null indicates a fatal error condition, stops testing and requires operator intervention. This error is only reported once, and is software latched by the sending peripheral. The action of reading the error code using CA clears the latch. The error flag is only reinitiated by repeating the attempted action and failing. Possible error codes are as follows: - CODE Description - null Clear and Working - K Test site EMPTY and no more product available.

	J OUTPUT FULL - Device or wafer cannot be put in the desired bin.
	all others Miscellaneous FATAL ERRORS (Application Specific)
Example	None

CD

Name	Request peripheral ID#
Applicable To	Probers and Handlers
Syntax	CD
Response	s
Description	The peripheral returns a string of variable length to 256 characters. Normally this includes the peripheral make and model numbers and any software revision level I.D.
Example	For an Electroglas 2001X wafer prober running revision K software: CD 2001X.K

CE

Name	Request test start
Applicable To	Probers and Handlers
Syntax	CE
Response	n[,n]
Description	If a null is returned, testing may not begin. Null is the sequence STX ETX with no text in between. Other numbers returned indicate which sites may be tested. For single site testing, the response will be a "1" for start test.
Example	To start sites 1,3 and 4: CE 1,3,4

CG

Name	Request XY position
Applicable To	Probers
Syntax	CG
Response	b1,b2,x,y
Description	This command returns the current XY die coordinate (x,y) and the status of the "End of Wafer" (b1) and "End of Product" (b2) flags. b1 is a 1 if the wafer is complete, 0 if not. b1 must become true after the last die on the wafer is binned. b2 is a 1 if there is no more product. b2 must be valid when b1 is true. Finally, x and y are the current die coordinates.
Example	None

CH

Name	Output full status
Applicable To	Handlers
Syntax	CH
Response	n[,n]
Description	This command causes the peripheral to return a list of full output tubes. If no outputs are full, a Null response is returned.
Example	None

CI

Name	Input empty status
Applicable To	Handlers
Syntax	CI
Response	n[,n]
Description	This command causes the peripheral to return a list of empty input tubes. If no inputs are empty, a Null response is returned.

Example

None

CU

Name	Request DUT/wafer ID
Applicable To	Probers and Handlers
Syntax	CU
Response	s
Description	This command causes the prober or handler to return an ASCII string denoting the ID numbers of the device or wafer currently under test in the test site.
Example	None

CZ

Name	Application specific status request
Applicable To	Probers and Handlers
Syntax	CZ
Response	s
Description	This command causes the peripheral to return an ASCII string for passing application specific status requests back to the tester.
Example	None

<

>

hpp849.8.17

<

>

P849 Communications Interface : Applications Layer : Bulk Data Transfer Commands

Bulk Data Transfer Commands

For bulk data transfer, data may be sent as various categories, such as wafer map or set-up data. Each category can be sent in the form of multiple blocks. Each block is comprised of multiple records that are passed as strings in the protocol. The bulk data transfer commands should not be interrupted by other commands. That is, once a command is started, it must be completed unless a time-out occurs.

DA

Name	Data block request to receive
Applicable To	Probers and Handlers
Syntax	DAc,n1
Response	n2
Description	This command requests the peripheral to send up multiple records of data to the tester. Each record is a string of up to 256 ASCII characters in length. Sending this command requests the peripheral to get ready to send. The actual transmission will be requested one record at a time by the DB command. The first argument c defines the type of data to be sent. The following data types are defined: - a: Wafer Map Data - b: Setup Data

- c: Alignment Setup Data
- d: Inspection Setup Data
- e: Error Log Data
- f: Yield Data

Within each data type there may be up to 256 blocks available. The second argument n1 defines the block number being requested for that data type. The numbers of blocks will be in the range 0 through 255. The block numbers must be used sequentially.

The peripheral will respond with the record count, n2 indicating how many records will be returned within that block. The number of records will be in the range of 0 through 255, where 0 indicates there are no records in that block, and the previous block was the last block.

Example

None

DB

Name	Request for next data record
Applicable To	Handlers and Probers
Syntax	DB
Response	s
Description	This command is sent to the peripheral to request the next data record to be transmitted. The peripheral responds with the data record s (256 character string). After receipt of the last data record, the tester should request a checksum (DC).
Example	None

DC

Name	Request checksum
------	------------------

Applicable To	Handlers and Probers
Syntax	DC
Response	n
Description	This command causes the peripheral to send the computed checksum for the data block which was just transmitted or received. The checksum is computed as the numeric sum of the unsigned binary values of all the bytes in all string locks (excluding STX and ETX characters). The checksum is sent as a decimal number in ASCII string format and is limited to modulo 65, 535 (overflow is ignored).
Example	To request the bulk data for setup block #1 in the peripheral: <pre>DAb,1 2 DB ABCDEF DB XYZ DC 672 DAb,2 0</pre> This example shows that there were two records to send in the first block. The checksum was computed to be 672, and there was no second block thus terminating the sequence.

DD

Name	Data block request to send
Applicable To	Handlers and Probers
Syntax	DDc,n1,n2
Response	c,n1,n2
Description	This command requests the peripheral to be prepared to receive data blocks that the tester is ready to send. The data is of type c where c is one of the following characters identifying data type: • a: Wafer Map Data

- b: Setup Data
- c: Alignment Setup Data
- d: Inspection Setup Data
- e: Error Log Data
- f: Yield Data

Each data type may comprise several blocks with n1 identifying the block number. The number n2 is the number of records to be sent. The peripheral acknowledges it is ready to receive by echoing the values c,n1,n2. If not ready, it returns a null string. n1 and n2 may each be in the range 1 through 255.

This command is generally used to return the data previously requested, received, and checked by the DA, DB, and DC commands.

Example

None

DE

Name	Send next data block
Applicable To	Handlers and Probers
Syntax	DEs
Response	n
Description	This command is used to send data blocks in records of up to 256 characters to the peripheral. It must have been preceded by a data block request to send (DD). The peripheral responds with the block number n, which may be in the range of 1 through 255. This should be followed by a check sum request (DC) after the last block has been sent.
Example	To request the bulk data for setup block #1 in the peripheral: DDb,1,2 b,1,2

DEABCDEF
DEXYZ
DC
672
This example shows that there were two records to send in the block.
The checksum was computed to be 672.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hpp849.8.18

<	>
-----------------	-----------------

P849 Communications Interface : Reference Section

Reference Section

The following topics are available:

- | | |
|---|-------------------------------------|
| • | ASCII Code |
| • | Sample Probing Flow |
| • | Sample Handler Flow |

<	>
-----------------	-----------------

hpp849.8.19

<

>

P849 Communications Interface : Reference Section : ASCII Code

ASCII Code

7-Bit ASCII Code					
Dec	Octal	Hex	Character	Control Equivalent	Description
000	000	00	NUL	@	NULL, CTRL SHIFT P, Tape Leader
000	001	01	SOH	A	Start of Header, SOM
002	002	02	STX	B	Start of Text
003	003	03	ETX	C	End of Text
004	004	04	EOT	D	End of Transmission, END
005	005	05	ENQ	E	Enquiry
006	006	06	ACK	F	Acknowledge
007	007	07	BEL	G	Bell
008	010	08	BS	H	Backspace, FEO
009	011	09	HT	I	Horizontal Tab, TAB
010	012	0A	LF	J	Line Feed, New Line, NL
011	013	0B	VT	K	Vertical Tab, VTAB
012	014	0C	FF	L	Form Feed, Form, Page
013	015	0D	CR	M	Carriage Return, EOL

014	016	0E	SO	N	Shift Out, Red Shift
015	017	0F	SI	O	Shift In, Black Shift
016	020	10	DLE	P	Data Link Escape, DC0
017	021	11	DC1	Q	XON, Reader On
018	022	12	DC2	R	Tape, Punch On
019	023	13	DC3	S	XOFF, Reader Off
020	024	14	DC4	T	Tape, Punch Off
021	025	15	NAK	U	Negative Acknowledge
022	026	16	SYN	V	Synchronous Idle, Sync
023	027	17	ETB	W	End of Text Buffer, LEM
024	030	18	CAN	X	Cancel, CANCL
025	031	19	ETB	Y	End of Medium
026	032	1A	SUB	Z	Substitute
027	033	1B	ESC	[	Escape, Prefix
028	034	1C	FS	\	File Separator
029	035	1D	GS	]	Group Separator
030	036	1E	RS	^	Record Separator
031	037	1F	US	-	Unit Separator
032	040	20	SP		Space
033	041	21	!		
034	042	22	"		
035	043	23	#		
036	044	24	\$		
037	045	25	%		
038	046	26	&		
039	047	27	'		Apostrophe
040	050	28	(		
041	051	29	)		
042	052	2A	*		

043	053	2B	+	Plus
044	054	2C	,	Comma
045	055	2D	-	Minus
046	056	2E	.	Period
047	057	2F	/	
048	060	30	0	
049	061	31	1	
050	062	32	2	
051	063	33	3	
052	064	34	4	
053	065	35	5	
054	066	36	6	
055	067	37	7	
056	070	38	8	
057	071	39	9	
058	072	3A	:	
059	073	3B	;	
060	074	3C	<	
061	075	3D	=	
062	076	3E	>	
063	077	3F	?	
064	100	40	@	
065	101	41	A	
066	102	42	B	
067	103	43	C	
068	104	44	D	
069	105	45	E	
070	106	46	F	
071	107	47	G	

072	110	48	H
073	111	49	I
074	112	4A	J
075	113	4B	K
076	114	4C	L
077	115	4D	M
078	116	4E	N
079	117	4F	O
080	120	50	P
081	121	51	Q
082	122	52	R
083	123	53	S
084	124	54	T
085	125	55	U
086	126	56	V
087	127	57	W
088	130	58	X
089	131	59	Y
090	132	5A	Z
091	133	5B	[
092	134	5C	\
093	135	5D	]
094	136	5E	^
095	137	5F	_
096	140	60	`
097	141	61	a
098	142	62	b
099	143	63	c
100	144	64	d

101	145	65	e
102	146	66	f
103	147	67	g
104	150	68	h
105	151	69	i
106	152	6A	j
107	153	6B	k
108	154	6C	l
109	155	6D	m
110	156	6E	n
111	157	6F	o
112	160	70	p
113	161	71	q
114	162	72	r
115	163	73	s
116	164	74	t
117	165	75	u
118	166	76	v
119	167	77	w
120	170	78	x
121	171	79	y
122	172	7A	z
123	173	7B	{
124	174	7C	
125	175	7D	}
126	176	7E	~
127	177	7F	DEL

<	>	
	2015 Teradyne, Inc. All rights reserved.	

hpp849.8.20

<	>
---	---

[P849 Communications Interface](#) : [Reference Section](#) : Sample Probing Flow

Sample Probing Flow

P849
Command

CD

AK

CA

AcS

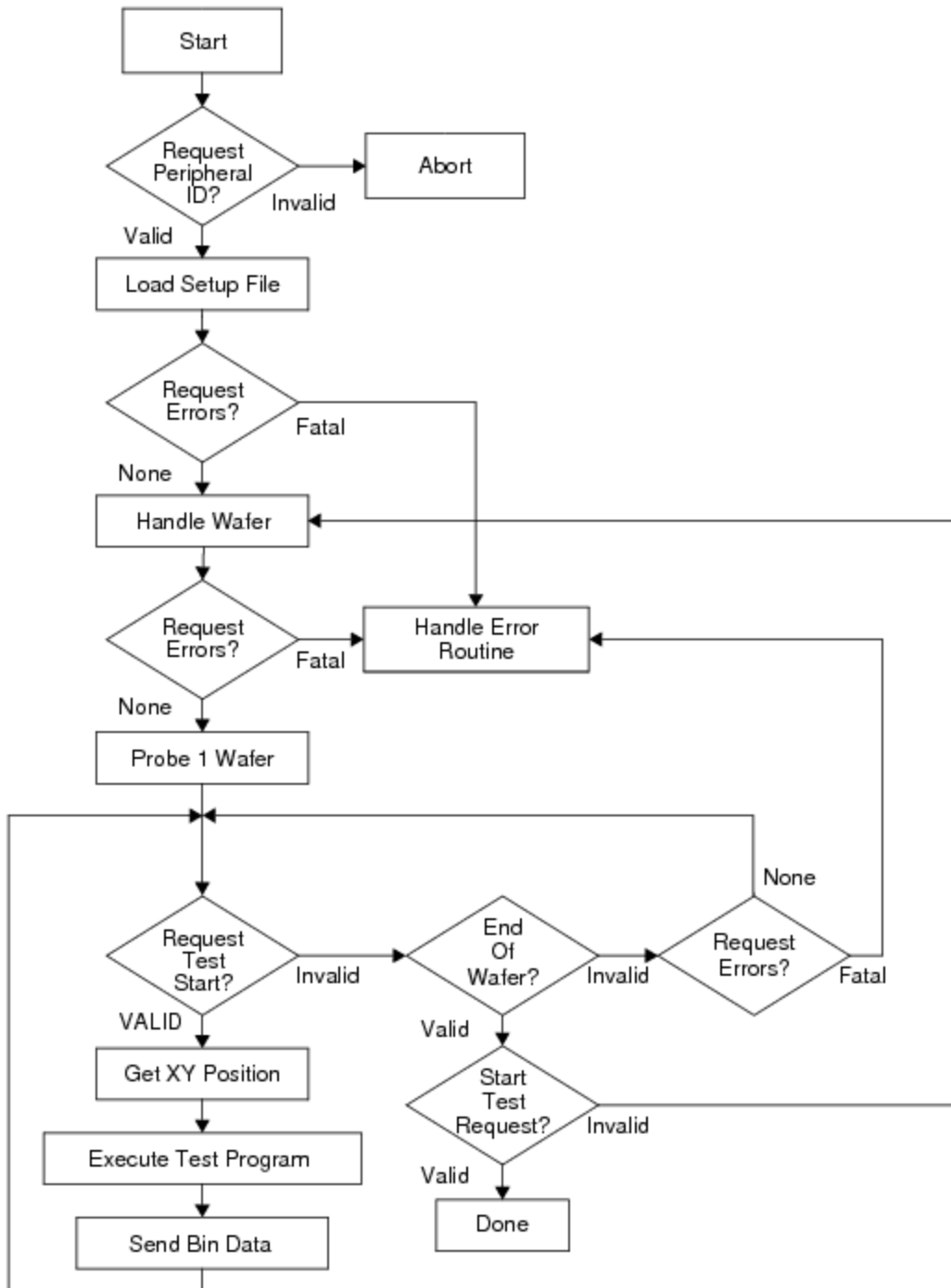
CA

AcL

CE

CG

BA



Sample P849 Probing Flow

<

>

hpp849.8.21

<

>

P849 Communications Interface : Reference Section : Sample Handler Flow

Sample Handler Flow

P849
Command

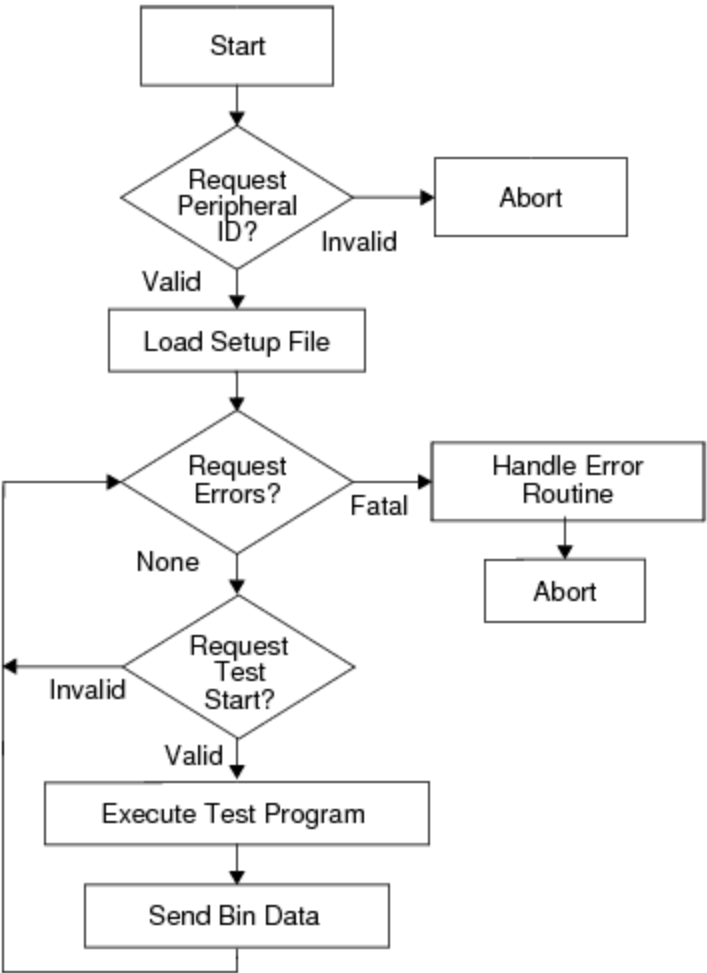
CD

AK

CA

CE

BA



Sample P849 Handler Flow

<	>	
	2015 Teradyne, Inc. All rights reserved.	

hpparallel.2.1

<	>
-----------------	-----------------

Parallel Interface

Parallel Interface

The parallel interfaces on most handlers and probers are custom to the manufacturer. Therefore, the UltraFLEX parallel interface is customizable to allow it to be used with almost any handler or prober. The ParallelInterface driver allows UltraFLEX tester users to communicate with a handler or prober using discrete TTL (Digital) signals. UltraFLEX testers use the National Instruments 32HS DIO boards which can sample incoming and generate outgoing digital signals per programmed specifications. The Teradyne-supplied ParallelInterface driver allows you to configure the signals to the desired timing, level, and duration without writing any code. The parallel interface is a separate option on J750, FLEX, and UltraFLEX testers.

The following topics are available:

- | | |
|---|---------------------------------|
| • | Hardware |
| • | Software |
| • | Important Notes |

<	>
-----------------	-----------------

hpparallel.2.2

<	>
-----------------	-----------------

Parallel Interface : Hardware

Hardware

UltraFLEX testers use the standard NI32HS Board and the SCB-68 breakout box. The NI32HS board requires no modification, but the SCB-68 does require minor hardware modifications. In addition, the prewired parallel port on the tester has a custom pinout but a standard D-sub connector. The National Instruments 32HS DIO board supports four physical ports. These ports can be used two at a time (A and B or B and C) or used four at a time, for either input or output operations. National Instruments does not recommend using a port for both input and output operations, and so some restrictions on how ports are configured are enforced within the Teradyne IG-XL driver. The other concept of “ports” is an output port or input port for IG-XL. Therefore, as used here, a “port” is the collection of pins used for a common function, such as writing bin/site data or reading test starts.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpparallel.2.3

<	>
-----------------	-----------------

Parallel Interface : Software

Software

UltraFLEX testers use the National Instruments software, NI-DAQ, that comes with the NI32HS board. NI-DAQ provides a generic programming interface that can be used to acquire or generate digital signals using the NI32HS and other NI Digital Acquisition cards. To simplify programming requirements, the ParallelInterface and any other custom parallel port drivers can use the NI driver through an IG-XL equipment interface. This allows you to use common handler/prober communication-oriented statements.

The following topics are available:

- [IG-XL Driver](#)
- [Flow](#)
- [Organization](#)
- [Functions](#)

<	>
-----------------	-----------------

hpparallel.2.4

<	>
-----------------	-----------------

Parallel Interface : Software : IG-XL Driver

IG-XL Driver

Unlike other IG-XL drivers, the same ParallelInterface driver is configurable for use with all handlers and probers instead of being designed for a specific model or manufacturer. Thus, developing a custom IG-XL Parallel driver is unnecessary. The Teradyne-supplied ParallelInterface driver includes a user interface that can store configurations in the Windows registry for multiple handlers and probers.

The following topics are available:

- | | |
|---|--|
| • | Configuration |
| • | ParallelInterface Input/Output Configuration |
| • | ParallelInterface TTL+RS232 Configuration |
| • | ParallelInterface Port Configuration |
| • | ParallelInterface Input Port Configuration |
| • | ParallelInterface Output Port Configuration |

Configuration

Select the ParallelInterface driver from an OI (SimpleOI, ProductionUI, or any custom UI) in the same way as any other handler/prober drivers using the EquipmentSelect control. When you

configure the driver (usually by selecting the Configure button on the EquipmentSelect control), the setup window shown in the figure is displayed.

Parallel Interface Driver Configuration Form

Input/Output Signals | TTL R3232 Combo | Pin Assignment Board 1

Input Signal Configuration (All Boards)

Input Signals:

- Start Signal Length (ms): 1
- EOL/ECW Signal Length (ms): 1
- Alarm Signal Length (ms):

Handshake:

- Enable Handshake Signal: ☒
- Signal Length (ms): 3
- Timeout (ms): 1000

Output Signal Configuration (All Boards)

ECS Sgna (End Of Test):

- Start Length (ms): 1
- Signal Length (ms): E

Data Signals (Bin Number/Site Number):

- Start Length (ms): C
- Signal Length (ms): E

Multi-Site Binning Options:

- ☒ Write bin results simultaneously for all sites
- ☐ Write bin results one site at a time

End of Test Delay Time:

Number of Milliseconds: 300

TT Signal Phase:

- ☐ High True
- ☒ Low True

Input Mask (Check if signal is used):

- ☒ Out of Product Line
- ☐ Alarm Line
- ☐ Feetest Line

Driver Setup:

Setup Name:

Save Setup As:

Save Setup in Registry: ☐

Number of Sites: 4

Assign Handler/Prober Site to Tester Site:

Tester	Equipment
0	1
1	2
2	3
3	4
4	

Buttons: Clear Configurations, OK, Cancel

IG-XL ParallelInterface Driver Configuration Window

You can save and select existing and new setups using the Driver Setup pane in the lower left-hand corner. This allows the same driver to interface with multiple handlers and probers by storing multiple driver setups. If you do not have IG-XL registry saving privileges, then the settings are only applicable for the current session until the driver is unloaded.

The Number of Sites box allows you to select the number of sites that the driver can support. The Assign Handler/Prober Site to Tester Site pane allows for sites to be mapped to equipment sites when they are not the same (for example handler site 1 is tester site 3). If no special mapping is required, select the default (tester site 0 to equipment site 1, tester site 1 to equipment site 2 and so on). Each site (based on the Number of Sites selection) must be defined in Assign Handler/Prober Site to Tester Site.

ParallelInterface Input/Output Configuration

At the top of the configuration screen are a series of tabs that allow for configuration of different aspects of the driver. The first tab, Input/Output Signals, allows you to define the signal characteristics. For input signals only the signal length can be specified. For output signals, however, you can specify the start (the length of time between the start of the output operation and the assertion of a signal) in addition to its length. Additionally, you can specify whether the signal is active High or Low (one setting for all signals). The Input/Output Signals tab is shown in the figure.

Input/Output Signals

TTI R5237 Emuln

Pin Assignment Board 1

Input Signal Configuration (All Boards)

Input Signals

Start Signal Length (ms)

1

EOL/EOW Signal Length (ms)

1

Alarm Signal Length (ms)

Handshake

Enable Handshake Signal

☒

Signal Length (ms)

0

Timeout (ms)

1000

Output Signal Configuration (All Boards)

EOS Signal (End Of Test)

Start Length (ms)

1

Signal Length (ms)

5

Data Signals (Bin Number/Size Number)

Start Length (ms)

0

Signal Length (ms)

5

TTI Signal Phase

☐ High True

☒ Low True

Input Mask (Check if signal is used)

☒ Out of Product Line

☐ Alarm Line

☐ Retest Line

Multi-Site Binning Options

☒ Write bin results simultaneously for all sites

☐ Write bin results one site at a time

End of Test Delay Time

Number of Milliseconds

300

IG-XL ParallelInterface Driver Input/Output Signals Tab

This tab is divided into several sections that include features of the driver. For some applications, not all values are required. You must fill in all required values, however, before saving a configuration to the registry or using it in the current session. The table lists the tab settings. All values are in milliseconds unless specified otherwise.

ParallelInterface Driver Input/Output Options				
Area	Signal	Option	Value	Notes
Input	Input	Start Signal Length	1	Length of test start or device ready signals from handler or prober. Required.
		EOL/EOW Signal Length	1	Length of End of Lot (Out of Product) and End of Wafer signal from handler or prober. Required.

	Handshake	Alarm Signal Length	1	Length of alarm signal. Optional.
		Enable Handshake Signal	On/Off	Enables and disables handshake response binning from handler or prober.
		Signal Length	1	Length of handshake signal from handler or prober. Required if enabled.
		Timeout	1	Maximum time to wait for handshake signal from handler or prober. Required if enabled.
Output	EOS/EOT	Start Length	0	Length of time to wait from start of binning cycle before asserting EOS/EOT signal. Required.
		Signal Length	1	Length of EOS/EOT signal. Required.
	Data	Start Length	0	Length of time to wait from start of binning cycle before asserting Bin signals. Required.
		Signal Length	1	Length of Bin signals. Required.
Multi-Site Binning	Bin	Write bin results simultaneously	On/Off	Writes all Bin signals for all sites at the same time.
		Write bin results one site at a time	On/Off	Writes Bin signals consecutively in site order.
End of Test Delay	EOS/EOT	Number of Milliseconds	0	Length of time to wait before starting the binning cycle. Required.
TTL Signal Phase	All	High True	On/Off	Selects all signals to be active high (nominal low).
		Low True	On/Off	Selects all signals to be active low (nominal high).
Input Mask	Input	Out of Product Line	On/Off	Enables Out of Product (End of Lot) callback function in driver or OI.

Alarm Line	On/Off	Enables Alarm callback function in driver or OI.
Retest Line	On/Off	Enables Retest functionality.

ParallelInterface TTL+RS232 Configuration

Since the parallel interface only supports TTL logic signals, it can be difficult to communicate other more complex information. To bridge this gap, the ParallelInterface driver has a mode of operation that allows it to work with the RS232 interface which, while much slower, does allow exchange of more detailed information. This mode requires the handler or prober to have an IEEE P849 (RDP) compliant serial interface. The tester must also have a serial cable.

In this mode, alarms and other special information are handled by sending a query to the handler/prober using the RS-232 interface. The information returned can include descriptions of error conditions, site mask updates, and X-Y coordinates. The TTL RS232 Combo tab allows you to configure the RS-232 component of the ParallelInterface driver.

Input/Output Signals

TTL RS232 Combo

Pin Assignment Board 1

RS232 Interface Configuration

Select COM Port

CCM1

Select Baud Rate

9600

Enable FLS Handshaking

☐

TTL-RS232 Combination

Use Alarm Line For Site Mask Updates

☐

Retrieve X/Y Coordinates (Probe)

☐

RS-232 Combo Tab, ParallelInterface Configuration Form

The setup for the RS-232 portion of the ParallelInterface is similar to the setup for the RS-232 interface by itself (see Configuration). The setup is simple and the defaults typically suffice. To integrate with the parallel interface, you must select two options based on the functionality required:

Use Alarm Line For Site Mask Updates	This check box forces the driver to poll the handler or prober for site information when the handler or prober signals the tester on the alarm line. Some handlers and probers allow this specialized use of the alarm
--------------------------------------	--

Retrieve X/Y
Coordinates

line but many do not. Before selecting this option, consult the handler or prober manufacturer’s documentation.

Configures the driver to automatically poll the handler or prober for the X-Y position of the DUT (site 0). While typically used for probers, this option can also be used for the newer strip handlers that maintain a map of devices on the strip. The driver first waits for a test start signal and then uses the RS-232 interface to retrieve the X-Y coordinate information before testing the device.

ParallelInterface Port Configuration

The Pin Assignment tab on the configuration screen allows you to configure the functionality of the lines on the 32HS DIO boards in the tester. Each port tab configures a different board. The typical interface only requires one NI32HS board, but up to three boards can be present if extended capabilities are required.

Input/Output Signals

TTI RS232 Command

Pin Assignment Board 1

Select Pins from the Available Ports

Click the right mouse button to configure the port when the selection has been completed.

Ports A and B (32HS Board)

Port A

DIOA7
Pin 15

DIOA6
Pin 48

DIOA5
Pin 47

DIOA4
Pin 13

DIOA3
Pin 12

DIOA2
Pin 45

DIOA1
Pin 44

DIOA0
Pin 10

Port B

DIOB7
Pin 22

DIOB6
Pin 21

DIOB5
Pin 54

DIOB4
Pin 53

DIOB3
Pin 52

DIOB2
Pin 51

DIOB1
Pin 17

DIOB0
Pin 16

Ports C and D (32HS Board)

Port C

DIOC7
Pin 28

DIOC6
Pin 61

DIOC5
Pin 30

DIOC4
Pin 26

DIOC3
Pin 25

DIOC2
Pin 53

DIOC1
Pin 57

DIOC0
Pin 23

Port D

DIOD7
Pin 34

DIOD6
Pin 67

DIOD5
Pin 36

DIOD4
Pin 32

DIOD3
Pin 31

DIOD2
Pin 64

DIOD1
Pin 63

DIOD0
Pin 29

Pin Assignment Tab, ParallelInterface Configuration Form

For the first NI-32HS board, Teradyne reserves the following lines for predefined signals:

- | | |
|-------|---------------------------|
| DIOB6 | EOS / EOT |
| DIOD0 | Start of Test |
| DIOD1 | Retest |
| DIOD2 | End of Lot / End of Wafer |

These lines appear shaded in cyan (light blue), and you cannot change their configuration. The ParallelInterface driver does not reserve output lines for any Binning lines. However, you can configure these and other signals from available unreserved output lines. If a handler or prober supports multiple Start of Test or End of Test lines, the ParallelInterface driver allows you to allocate additional lines for these purposes (see ParallelInterface Input Port Configuration). You cannot delete or move the reserved lines.

For configured ports, place the pointer over the block representing the line and a tooltip text displays the current function of the line. For lines reserved by the driver, the tooltip text also includes the fixed bulkhead connector pin to which a line is connected. Note that the Pin designation displayed (for example DIOA7 Pin 15) is the pin number in the SCB-68 box, not the bulkhead cable.

Configure lines by clicking the desired boxes. Once selected, a line's box changes color to amber. Deselect lines by clicking them again, and the color returns to white. Select multiple-line ports a group by clicking each line separately.

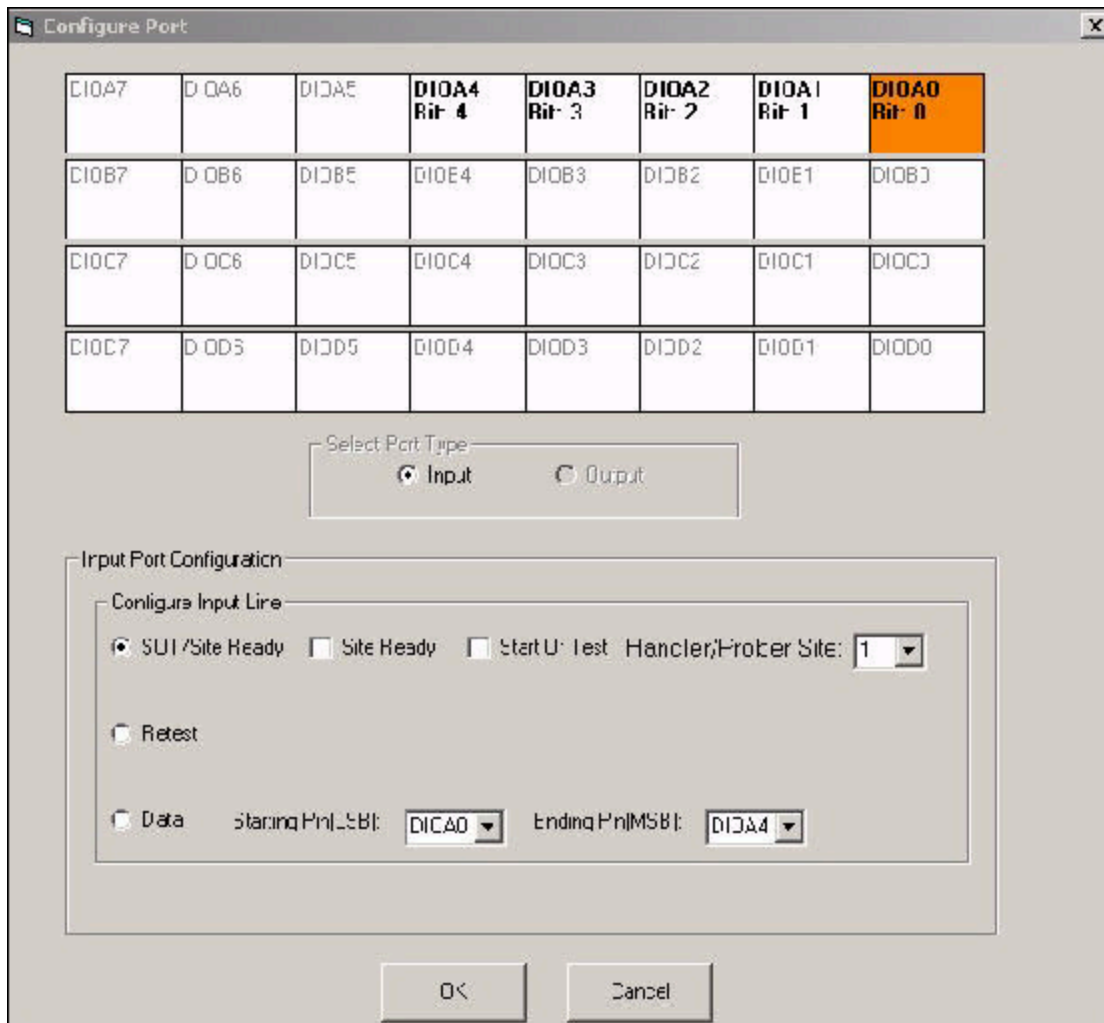
When you have selected lines, you can position the pointer over one of the selected lines (or a single line), right click, and select **Modify Port** to bring up the port configuration menu. The Configure Port window allows you to configure the function of the selected lines. The menu also has items that allow you to modify a port (or line) and to delete a port.

Note that for output, other than of EOS lines, there are typically not any single line ports. Ports tend to be multiline entities. You can also select **Delete Port** to unprogram an entire port.

IG-XL includes consistency checks to make sure that an input line and an output line do not reside within the same physical port or those lines belong to a single port and there is no overlap. With multiboard interfaces, NI32HS ports are allocated for either input or output operations (but not both) as a performance enhancement. Any illegal selections are warned against and cannot be saved.

ParallelInterface Input Port Configuration

The figure shows the Configure Port window. The port is highlighted in bold and, in the case of a multiline port, each line is assigned a symbolic bit number. When the previously selected port is selected to be Input, the bottom pane changes to allow for input configuration. Since each port is configured at the same time, the configuration applies for all the lines that were previously grouped into a port.



Configure Port Window Showing Input Port Configuration Pane

In the Input Port Configuration pane, only a few options are available for input lines and ports. This is because most of the configuration is done on the first tab when the timing is set. However, here you can select the function that IG-XL uses to react to the input: SOT, Retest, or generic data. For the SOT/Site Ready, you must select if the line is a start and/or site ready and for which sites (one particular or all). Typically, the start line and site ready lines are separate on a handler or prober. In the event they are shared, however, the driver can be configured to accommodate.

Choosing the Retest functionality forces the device to be treated as a retest device in the datalog. The Data option allows for a user-defined reception of information. Most handlers and probers do not offer extended information. If they do, however, you can configure the driver to receive this information.

Note that if the information is to be used by the tester, then the IG-XL ParallelInterface driver must be modified since no default functionality exists. Other inputs (such as EOW and EOL/Out of Product) are dedicated and are not reconfigurable (see ParallelInterface Port Configuration).

ParallelInterface Output Port Configuration

This figure shows the Configure Port window with the Output Port Configuration pane. The port is highlighted in bold and, in the case of a multiline port, each line is assigned a symbolic bit number. When the previously selected port is selected to be Output, the bottom pane changes to allow for

output configuration. Since each port is configured at the same time, the configuration applies for all the lines that were previously grouped into a port.

Configure Port

DIOA7	DIOA6	DIOA5	DIOA4 Bit 4	DIOA3 Bit 3	DIOA2 Bit 2	DIOA1 Bit 1	DIOA0 Bit 0
DIOB7	DIOB6	DIOB5	DIOB4	DIOB3	DIOB2	DIOB1	DIOB0
DIOC7	DIOC6	DIOC5	DIOC4	DIOC3	DIOC2	DIOC1	DIOC0
DIOD7	DIOD6	DIOD5	DIOD4	DIOD3	DIOD2	DIOD1	DIOD0

Select Port Type
☐ Input ☒ Output

Output Port Configuration

Data Type

☒ Bin # ☐ Site # ☐ End of Test ☐ Site Disable Equipment Site # 1

Data Encoding

☐ Binary Encoding ☒ Bit Shift

Starting Pin (LSB): DIOA0 Out Of Range Value:

Ending Pin (MSB): DIOA4 Value Represented by LSB:

OK

Cancel

Configure Port Window Showing Output Port Configuration Pane

First, in the Data Type area, you select the function of the port and the site numbers to which this port applies (one or all sites). The options are:

Bin	The port sends binning information to the handler/prober.
Site	The port sends the site number (as selected on the far right) to the handler. This is typically used with binning information that allows the handler or prober to determine to what site the bin data applies.
End of Test	The port sends a single End of Test (or End of Sequence) pulse to the handler or prober based on the settings on the main configuration form.
Site Disable	Disables sites on the handler from further testing (not applicable to probers in most cases).

After selecting the function of the port, select the type of encoding to use. Specify the encoding (bit shift or binary) and which lines correspond to the LSB (least significant bit) and MSB (most significant bit). Reverse encoding (LSB being the higher order bit/line) is also supported. The table shows examples of how the bin lines would be activated (assuming five bin lines read right to left) using the two different binning schemes.

ParallelInterface 5-Bit Bin Encoding Examples

Bin Number	Binary Encoding	Bit Shift Encoding
3	00011	01000
1	00001	00010
21	10101	Overflow

Selecting Binary Encoding means that the port data is expressed as a binary number. For example, data of "21" decimal would be expressed as "10101" binary over the port's bits. Bit Shift data expresses the data where each binary digit represents a decimal number based on its place. For example, data of "3" decimal would be expressed as "00100" binary over the port's bits (assuming LSB Value of 0 as explained below).

To aid in error or out of range situations, two options allow sending a preselected value:

Out of Range	The value that will be sent by the handler if a value that is out of the port's range is sent to it. For example, if a 4-bit port is selected with and Out of Range value of "0" and if the test program bins a device to bin 20, the driver would bin it to bin 0 automatically.
Value Represented by LSB	Allows you to set the minimum number. Some handlers and probers do not accept bin 0, so the LSB can be preselected to another bin number to avoid this scenario.

<

>

2015 Teradyne, Inc. All rights reserved.

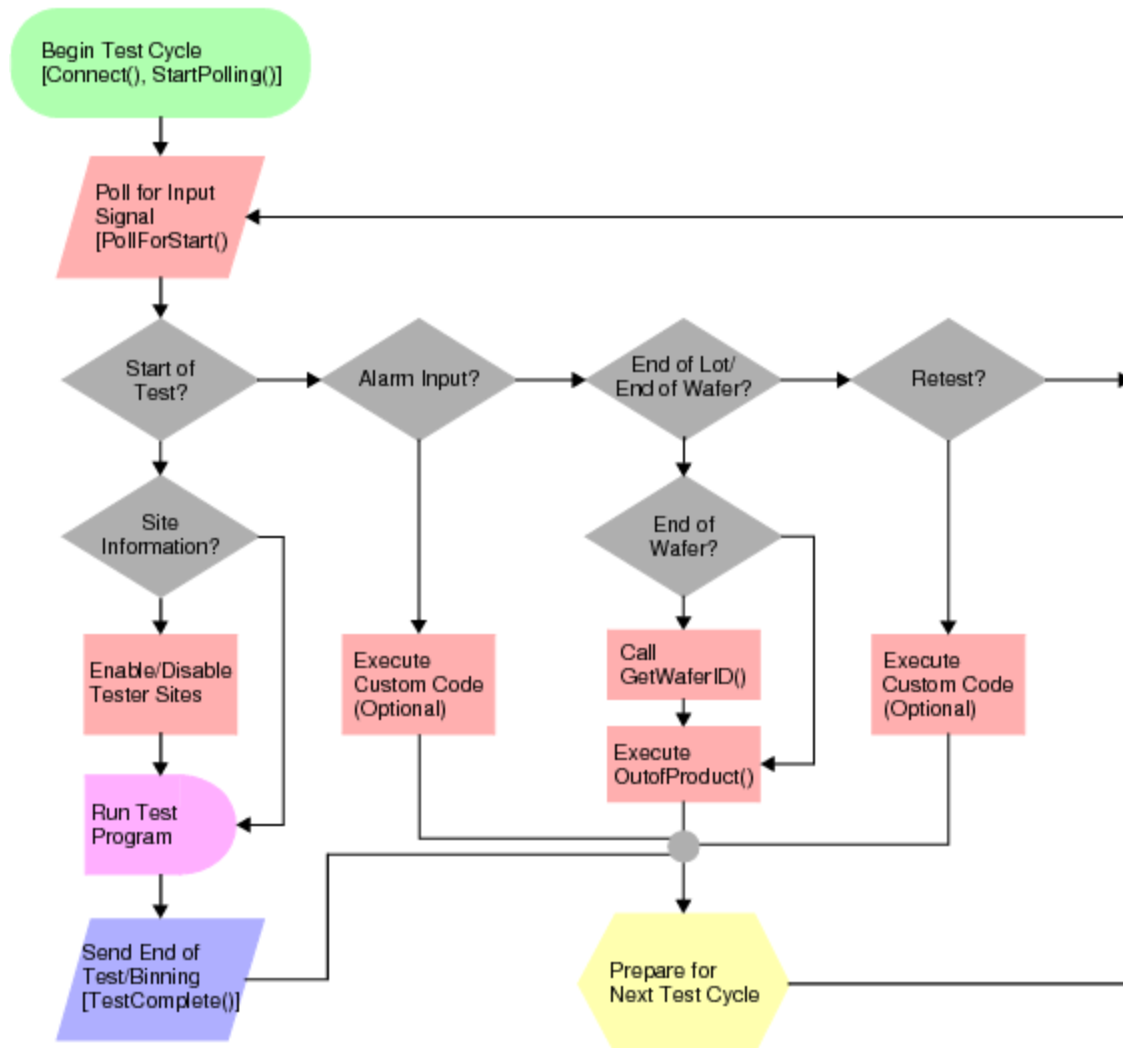
hpparallel.2.5

<	>
---	---

Parallel Interface : Software : Flow

Flow

For each test cycle, the following figure shows the flow of the ParallelInterface driver. The contents of the called functions are custom, but the driver processes devices through the tester in the same order. Note that Start of Test, Alarm, End of Lot /End of Wafer and Retest are mutually exclusive.



ParallelInterface Driver Test Cycle Flow

If you use the Teradyne-supplied ParallelInterface driver, this flow explicitly applies. For other custom drivers, the flow can differ. The important point is that at the beginning of the cycle, all inputs are processed and at the end of the cycle, all outputs from the tester are sent. Inputs during the test program run are not acted on until the beginning of the next test cycle.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hpparallel.2.6

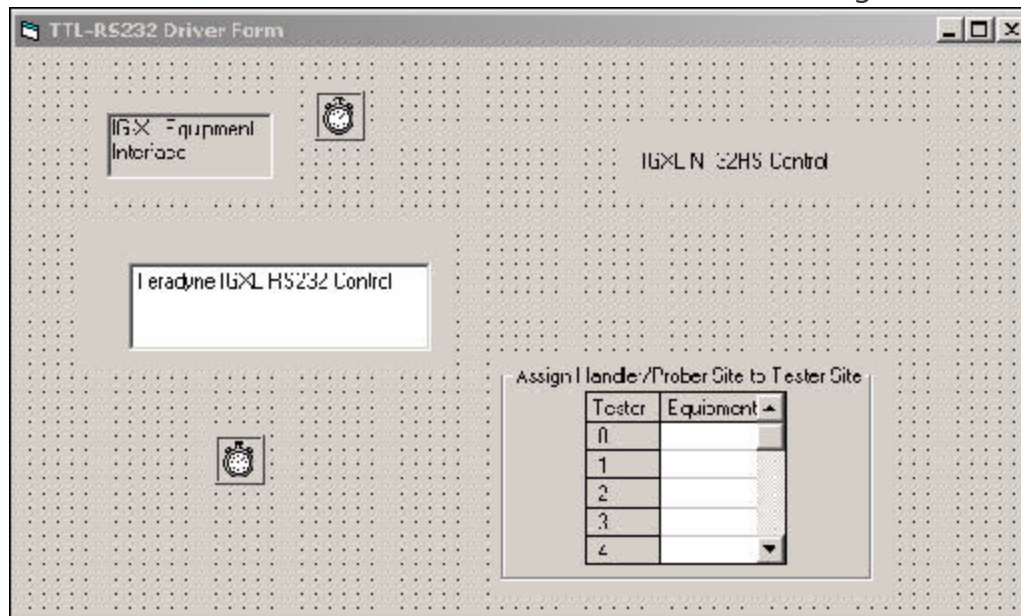
<

>

[Parallel Interface](#) : [Software](#) : Organization

Organization

The unique part of the [ParallelInterface](#) driver is the IGXLNI32HSControl object on the driver form. (For a description of the other standard elements found on a Parallel driver, see [Operator Interface: EquipmentSelect](#).) IGXLNI32HSControl contains functionality that makes it easier to create a parallel interface driver to use in multiple applications without writing NI32HS hardware-level code. In most instances, the Teradyne-supplied ParallelInterface driver provides all the functionality required and, therefore, no code must be written. If you use a custom Parallel driver, the following properties, events, and methods help create a viable driver. The main form of the Teradyne-supplied ParallelInterface driver with all its elements is shown in the figure.



ParallelInterface Driver Main Form

Note that the [ParallelInterface](#) driver, in addition to the required elements, has a second timer for handshake binning and support for site mapping. While optional elements are not required, they add additional features to the driver.

Properties

The table lists the properties for the IGXLNI32HSControl object. The Read/Write status of the property is listed in the two rightmost columns. Properties that are read only can only be read by the driver, whereas the properties that are both Read and Write can be altered.

Properties for the IGXLNI32HSControl Object

Name	Type	Description	Design Time	Run Time
DisplayErrorMessages	Boolean	Displays error messages that occur if selected	R/W	R/W
EOSPinReserved	Boolean	If selected EOS pin cannot be changed or removed	R/W	R/W
NumberOfBoards	Integer	Contains the number of NI32HS boards installed in the tester	N/A	N/A
RetestPinReserved	Boolean	If selected Retest pin cannot be changed or removed	R/W	R/W
SignalsActiveLow	Boolean	If selected makes all signals to be active low true	R/W	R/W

These properties allow you to configure some of the basic functions of the IGXLNI32HSControl object such as if the reserved pins are fixed, if error messages are to be displayed, and the active state of the signals. The IGXLNI32HSControl object automatically detects the NumberOfBoards and this property is not public. Note that if SignalsActiveLow is selected, then a hardware change may be required. Consult the Teradyne ParallelInterface option installation manual to determine if the SCB-68 unit attached requires a modification or if it is auto-setting. Early SCB-68 units do not have Teradyne active circuitry that allows the bias to change using hardware selection.

Events

The IGXLNI32HSControl object has no public events. Lot/product control events (such as End of Lot and End of Wafer) are implemented by using the EquipmentInterface or EquipmentSelect methods (AssertOutOfProduct, AssertEndOfWafer, and so on).

Methods

The table lists the methods available for the IGXLNI32HSControl object. Each function performs a specific action when one of the events occurs.

Methods for the IGXLNI32HSControl Object

Name	Type	Arguments	Description
ApplyConfigurations	Boolean	[OverridePortCDIO As Boolean = False]	
ClearConfigurations	N/A	(none)	
ClearInputOperation	N/A	(none)	
ClearLineMasks	Boolean	(none)	
ConfigurePort	Integer	PortType As Integer portFunc As Integer startPin As Integer endPin As Integer encoding As Integer signalStart As Integer signalLength As Integer LSBVal as Integer OutOfRangeVal As Integer PortName as String	Configures a port based on the LSB/MSB, received as line numbers, and from other characteristics. The range of lines is checked against other ports to verify that there is no overlap with other ports. If there is overlap, than an error is flagged.
DisplayErrorMessage	N/A	Emsg As String [displayMessages As Boolean = True]	
GetPortDataValue	Long	PortID As Integer	
InitiateRead	Boolean	(none)	
InitiateWrite	Boolean	(none)	For multiboard systems, the slave boards are set up to write when the master board write operation begins. Data buffers are passed to the boards and the write operation is initiated on the master board. For single board systems, the data is written to the

board, and the write operation
Immediately starts. If handshaking is enabled, then an input operation is initiated concurrently to get wait for the handshake line. The driver waits until the handshake line is asserted or until the handshake time out expires. If the handshake is not received within the specified time, an error is flagged. When the output operation is complete the output array is cleared, while the handler or prober is indexing.

InputReceived	Boolean	PortIDs() As Integer numPids As Integer	
ReadInputPortTTL	Integer	PortIDs() As Integer sizeOfPids As Integer	
ResetBoard	Boolean	boardNumber As Integer	Allows you to reset a board.
SetupReadOpn	Boolean	PortID As Integer	
SetupWriteOpn	Boolean	portID As Integer datavalue As Integer	
StartOutputOperation	N/A	boardNum As Integer GroupNum As Integer	

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpparallel.2.7

<

>

Parallel Interface : Software : Functions

Functions

The functions described here are unique to the ParallelInterface driver and are not part of the IGXLNI32HSControl object. Therefore, they are only used in the ParallelInterface driver.

Function	ConfigurePort
Synopsis	Private Function ConfigurePort(boardNum, site, portType, encoding, function, LSB, MSB, OutOfRangeVal) As Boolean
Description	This function is called once you specify the configuration (or it has been read from the registry) and it has been validated. The function calls the NI32HSControl.ConfigurePort() function with the appropriate arguments and receives a portID. The port ID is saved in the structure for this site. For output ports, it is placed in an element according to the function (EOS, Bin Data, or Site Data). If it is an input port, it is placed in the list of input ports. This function is called during configuration. If an error occurs an error message is displayed and the configuration ends.
Parameters	boardNum: An integer value defining the board number where the lines are located. site: For output ports, the site number for this port. For input ports this is used to indicate which if this is a SiteReady flag or StartOfTest Flag for that site. portType: Input or Output Encoding Bit Shift or Binary. Irrelevant for input ports.

	function: For output ports, this is EOS, Site Data, or Bin data. For input ports this is Alarm, Handshake, EOL/EOW, SOT, SiteReady, Data, Retest, LSB: Starting bit/line number MSB: Ending bit/line number OutOfRangeVal: Optional, used for output. This is the value that is written if the field cannot support a data value.
Return Value	True if the port configuration succeeded otherwise False

Function	WriteBinData
Synopsis	Private Function WriteBinData(Bin) As Boolean
Description	Writes bin/site/EOS data to the 32HSDIO board. This function loops through each of the elements in the bin array, site by site. As each element is accessed, the corresponding output configuration for that site is also accessed. An NI32HSControl.writePort() function is called with the bin number (and portID). If there is a site data portID, then the site number is passed to NI32HSControl.writePort(), along with the port ID. As far as EOS goes, every site will have an EOS portID, even though all sites may share the same EOS. When writing the EOS, the NI32HSControl.writePort() function is called passing it a value of 1, as the EOS is a single line. Although the ParallelInterface attempts to write the EOS for every site, if the EOS line is shared, the writePort function returns without writing if the port has been written already. Finally, when all sites are written, NI32HSControl.InitiateWrite() is called. When the write operation completes the input operation is initiated if the user has not disabled the driver.
Parameters	Bin(): An array of bin numbers. This array is Indexed by site number.
Return Value	True if the operation succeeded.

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hpparallel.2.8

<	>
-----------------	-----------------

Parallel Interface : Important Notes

Important Notes

The ParallelInterface driver operates like any other handler/prober driver interfacing to the rest of IG-XL through the ProductionControls.ocx library. In hardware, one or more 32HSDIO boards can be added to the tester’s computer assuming there is an open PCI slot. The number of boards (and hence TTL I/O pins) that the tester can support is limited by the number of available slots. On the software side, the National Instruments DAQ software and libraries must be installed as part of the ParallelInterface kit installation. This software is used to enable the driver to communicate with the NI-DAQ boards. A possibility for conflict between the DAQ software and other National Instruments products does exist. Only install the Teradyne-recommended driver versions to avoid any such conflict.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hprs232.4.01

<	>
----------------------	----------------------

RS-232 Interface

RS-232 Interface

One of the simplest interfaces to design, debug, and use is the serial RS-232 interface. The tester comes preprogrammed with a serial interface that is specifically designed for ATE use. Alternatively, you can design a custom RS-232 interface using a set of core RS-232 functions in the IG-XL RS232Control object.

The following topics are available:

- | | |
|---|---------------------------------|
| • | Hardware |
| • | Software |
| • | IG-XL Driver |
| • | Important Notes |

<	>
----------------------	----------------------

hprs232.4.02

<	>
-----------------	-----------------

RS-232 Interface : Hardware

Hardware

IG-XL-based testers use the standard serial port that is part of the tester’s computer. No additional cards or hardware are required to drive the serial port. The serial port uses a FIFO buffer that is UART 16550 compatible. The serial port settings are all made in software, so no hardware installation or configuration is required. Standard RS-232 cables (pinout and connectors) are used external to the tester.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hprs232.4.03

<	>
-----------------	-----------------

RS-232 Interface : Software

Software

IG-XL-based testers use the standard Windows NT/2000 serial port driver (WINNT/system32/DRIVERS/serial.sys). All configuration is done in the IG-XL driver. An RS-232 handler/prober driver is prewritten to support IEEE P849 (also known as RDP), the common RS-232 communications standard used in the ATE industry. The driver, called P849, can be loaded using the EquipmentSelect control menu. The existing driver is fully P849 compliant and can be used as-is to interface with all P849 compliant handlers and probers. You can also modify it to add additional functionality. If you need a custom RS-232 interface, IG-XL provides the tools to create one. The following topics are available:

- | | |
|---|--|
| • | P849 Protocol |
| • | RS-232 Native Mode Protocols |

<	>
-----------------	-----------------

hprs232.4.04

	
---	---

RS-232 Interface : Software : P849 Protocol

P849 Protocol

The tester supports a serial interface that uses standard RS-232 cabling and hardware in addition to the IEEE p849 communications protocol. The IEEE P849 protocol, more commonly known as RDP, was developed by Teradyne to address the shortcomings of parallel interfaces. Most handlers and probers support the IEEE P849 protocol as a standard for ATE communication. RDP is a command set that supports the most common communications needs as well as the ability to send or create a custom command. For a description of RDP, see [P849 Communications Interface](#). This section describes the P849 protocol and all messages and responses.

The major advantage of using a serial interface is that RDP is a secure protocol. Every transaction is prefaced with a Request to Send (ENQ) that must be responded to with an Acknowledge (ACK) or Not Acknowledge (NAK), and every message is enclosed in a Start of Text (STX) and End of Text (ETX) envelope. If an error condition violates the protocol, the message can be resent if necessary. If data integrity is a major concern, RDP ensures that the correct information is communicated.

In addition to being secure, a serial interface is also flexible. Any command, part of the standard RDP set or a custom command, can be sent to the handler or prober. Since most of RDP is a fixed standard, little configuration is required to set up a serial interface and the cable is standard.

Another important point is that the same interface works for single site and multisite testing due to the way that RDP is written. Therefore, to get a tester and handler or prober communicating and running production in the shortest amount of time, a serial interface is the interface of choice.

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hprs232.4.05

<	>
-----------------	-----------------

RS-232 Interface : [Software](#) : RS-232 Native Mode Protocols

RS-232 Native Mode Protocols

While the Teradyne-supplied driver does not support non-RDP RS-232 serial communications, there is no reason that a custom driver could not. Several handler and prober manufacturers, in addition to supporting RDP, also have their own native RS-232 communications languages. These protocols are standard RS-232 serial protocols that have specialized messaging based on a manufacturer’s requirements.

The advantage of these protocols is that they often give increased functionality and flexibility to the interface by offering extended control of the handler or prober. RDP is limited to the core set of ATE functionality that is required to test devices and, due to its structure and control, can be less efficient than some of the OEM RS-232 serial drivers. See [Organization](#) for information on Teradyne-supplied components and functions that can be used to create a custom native-mode RS-232 driver.

A disadvantage of these drivers is that they are all different. Typically, each manufacturer creates a unique driver that is not similar or applicable to another manufacturer’s piece of equipment. Historically, therefore, most RS-232 applications have used RDP. For intricate custom control, GPIB is recommended due to its superior data rate. For more information, see [GPIB Interface](#).

<	>
-----------------	-----------------

hprs232.4.06

<	>
-----------------	-----------------

RS-232 Interface : IG-XL Driver

IG-XL Driver

Each serial driver can be unique; however, IG-XL provides a set of standard tools for accomplishing communications. For most applications the existing P849 driver is sufficient and no customization or coding is necessary. If a new or modified RS-232 driver is required, however, this topic describes the IG-XL Visual Basic driver components and functions.

The following topics are available:

- | | |
|---|---------------|
| • | Configuration |
| • | Flow |
| • | Organization |
| • | Functions |

<	>
-----------------	-----------------

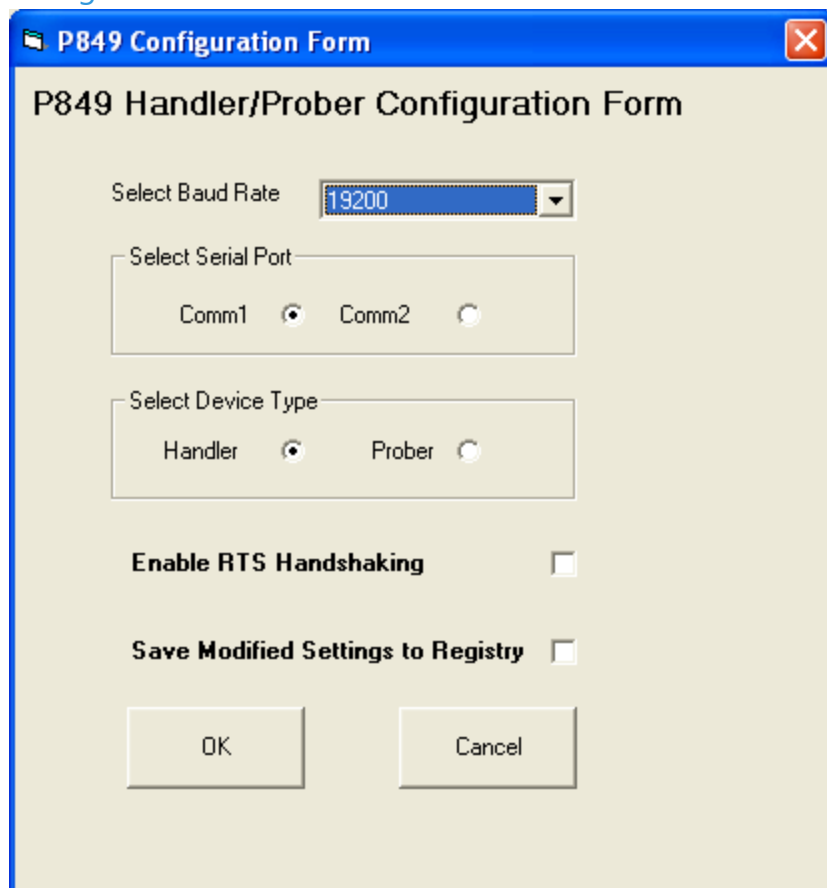
hprs232.4.07

<	>
----------------------	----------------------

RS-232 Interface : IG-XL Driver : Configuration

Configuration

The P849 driver has a configuration form that lets you select options to configure the driver's mode of operation. Since P849 is a tight standard, few options are available. The figure shows the configuration form for the P849 driver.



The screenshot shows a Windows-style dialog box titled "P849 Configuration Form". Inside, the title "P849 Handler/Prober Configuration Form" is displayed. The form contains several configuration options: a "Select Baud Rate" dropdown menu set to "19200"; a "Select Serial Port" section with radio buttons for "Comm1" (selected) and "Comm2"; a "Select Device Type" section with radio buttons for "Handler" (selected) and "Prober"; an "Enable RTS Handshaking" checkbox which is unchecked; and a "Save Modified Settings to Registry" checkbox which is also unchecked. At the bottom of the dialog are "OK" and "Cancel" buttons.

P849 RS-232 Driver Configuration Form

Communications options available are:

Baud Rate	The baud rate is the data communications rate between the tester and the handler or prober. Baud (bits per second) is in preset increments and is the maximum data rate. The tester supports 300, 1200, 2400, 4800, 9600, and 19200 baud. Most handlers and probers support a maximum of 9600 baud.
Serial Port Selection	The tester has two serial ports, both of which are identical. The tester can use either one for communicating with a handler. Naturally, you must plug the handler into whatever port you select on the tester. Each port is configured to communicate at the specified baud rate and N-8-1 (No Parity-8 Data Bits-1 Stop Bit) which is the industry standard for RS-232 serial handler/prober communications. Other RS-232 data configurations (such as E-7-1 and O-8-0) require modifying the IG-XL driver code.
Device Type	The tester can communicate with either a handler or a prober using the same P849 driver. If you select Prober, then the tester polls the prober for X-Y Coordinates at the beginning of every test program run in the PollForStart() function. Otherwise, Handler and Prober perform the same communications.
RTS Handshaking	Selecting the Enable RTS Handshaking checkbox forces the tester to use RTS (Request To Send) hardware handshaking with the handler or prober. RTS is a hardware line on the serial cable. A few older handlers and probers require this, but most of the time it is unnecessary.

Note that the Teradyne-supplied RS-232 driver conforms to RDP. Thus, no configuration is necessary for the messaging layer. Also, since RS-232 governs the hardware settings, no other hardware or software settings are necessary to configure RS-232 communications on IG-XL-based testers.

		
	2015 Teradyne, Inc. All rights reserved.	

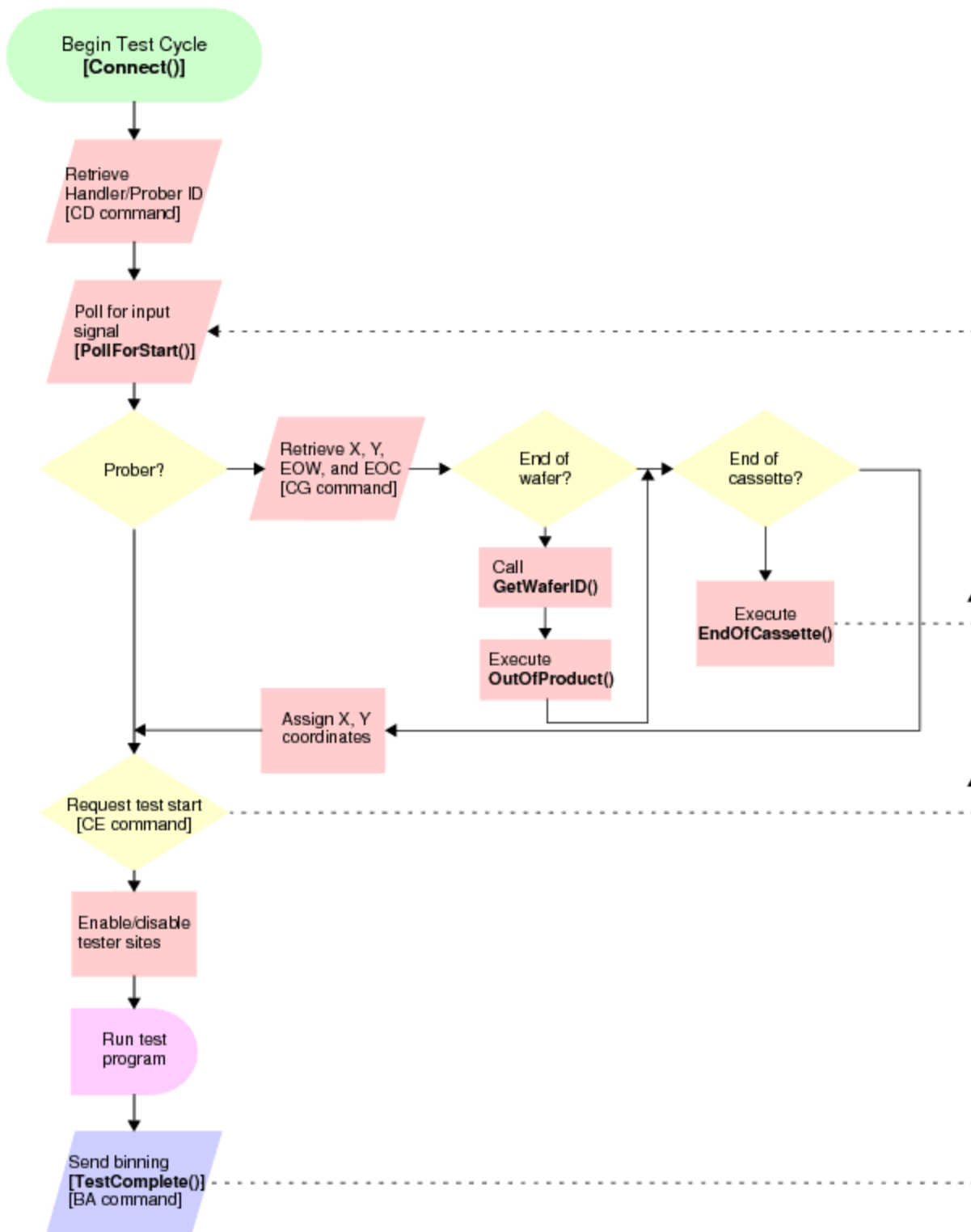
hprs232.4.08

<	>
---	---

RS-232 Interface : IG-XL Driver : Flow

Flow

The P849 driver has a set flow that it goes through for every iteration of the test cycle. The flow for a handler or prober differs slightly, but the core is essentially the same. While there are many other RDP commands available, IG-XL-based testers only require the use of a few key commands to accomplish testing. This is shown in the following figure.



P849 Driver Test Cycle Flow

Note that each section of the test cycle flow for P849 communications has a fixed command (such as CE, BA, or CG) unlike the other communications protocols. This is because of the P849 command set that is enforced. Each command performs a specific task that is governed by the IEEE P849 standard. For more information, see P849 Communications Interface.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

hprs232.4.09

<

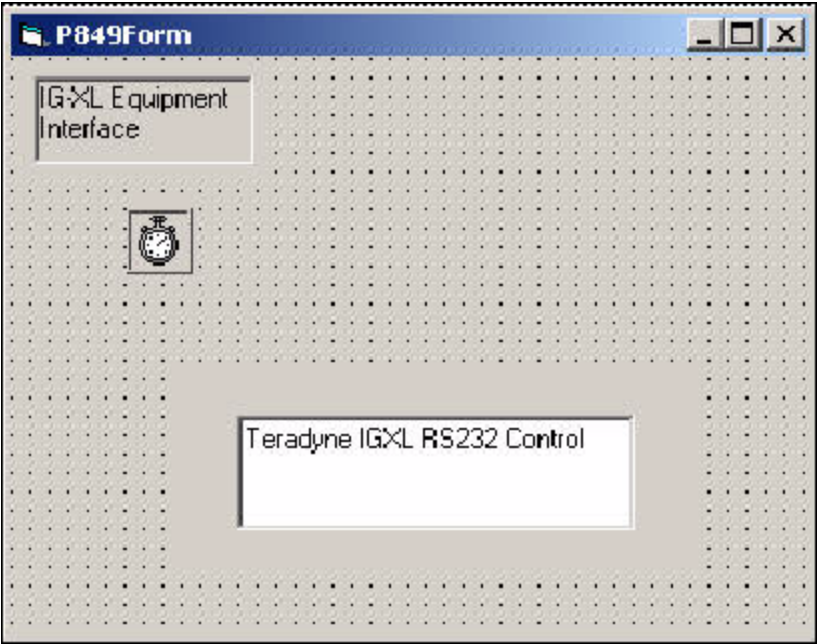
>

RS-232 Interface : IG-XL Driver : Organization

Organization

The unique part of the RS-232 driver is the IGXL RS232Control object on the driver form. Like other IG-XL communications drivers, a form is created that contains the required elements for communications. The figure shows an example RS-232 driver (the Teradyne-supplied P849 driver). The following topics are available:

- [Properties](#)
- [Events](#)
- [Methods](#)



RS-232 (P849) Driver Main Form

Properties

The table lists the properties for the RS232Control object. The Read/Write status of the property is listed in the two rightmost columns. Typically most properties are set from the configuration form inside the driver. However, some options need not be configured in normal operations such as Parity and DataBits.

Properties for the RS232Control Object

Name	Type	Description	Design Time	Run Time
BaudRate	Integer	Baud rate of the connection: 300, 1200, 2400, 4800, 9600, or 19200. Default is 9600 baud.	R/W	R/W
CommPort	Integer	Number of the RS232 port (on the computer frame) into which the device is plugged in. Choice of 1 for Comm1 or 2 for Comm2	R/W	R/W
DataBits	Integer	Number of data bits from 1 to 8 (default)	R/W	R/W
InputLen	Integer	Number of characters read from the bus at a time, usually 1	R/W	R/W
InputMode	Integer	Format of data read over the RS-232 line, 0 for Text (default) or 1 for Binary	R/W	R/W
InterCharTimeout	Integer	Time to wait between received characters in milliseconds	R/W	R/W
Parity	Integer	Choice of 0 for Even or 1 for Odd	R/W	R/W
RTSHandshake	Integer	Specifies whether RTS hardware handshaking is to be used (uncommon). Choice of 0 for disabled or 1 for enabled.	R/W	R/W
SimulatorMode	Boolean	Determines whether the driver uses the RS-232 bus or the internal simulator for communications	R/W	R/W
StartStopBit	Integer	Specifies whether a start or stop bit is to be used. Choice of 0 for disabled or 1 for enabled.	R/W	R/W

Events

This table lists the Visual Basic events that the RS232Control can detect. Handler/prober events (such as End of Wafer and End of Lot) are detected by the EquipmentSelect control in the Operator Interface or by the EquipmentInterface control on the driver form. In the case of the RS232Control object, all events are simulator events and are, therefore, not typically used during production testing.

Events for the RS232Control Object

Name	Arguments	Description
RS232Initialize	(none)	Raised if SimulatorMode is TRUE to signify that the RS-232 interface is being initialized. Calls RS232Initialize method.
RS232Read	str As String strLen As Integer	Raised if SimulatorMode is TRUE indicating the RS232 control is reading data from the device. Instructs the simulator to send data to the driver.
RS232Write	str As String strLen As Integer	Raised if SimulatorMode is TRUE indicating that the control is writing data to the device. Instructs the simulator to read data from the driver.

Methods

This table lists methods available for the RS232Control object. Each function performs a specific action when one of the events occur. Since the events are all simulator events, the methods are likewise simulator-only methods.

Methods for the RS232Control Object

Name	Type	Arguments	Description
InitializeRS232	N/A	(none)	Initialize the RS232 link between the tester and a handler or prober

hprs232.4.10

<

>

RS-232 Interface : IG-XL Driver : Functions

Functions

The following functions are part of the RS232Control object and can be used optionally in any serial handler/prober driver:

- RS232Control Functions
- P849 Functions

The default P849 driver uses the core functions and has a selection of functions that are available for you to modify. P849-specific functions are optimized to read and write data over the RS-232 interface using the RDP protocol and syntax. Any custom native-mode drivers may or may not use RDP-like constructs.

RS232Control Functions

This table lists the RS232Control object functions used to access the RS-232 hardware in the tester. Each function can be used at any time in the driver to communicate with a handler or prober. Since they are part of the RS232Control object, they can be used by any driver that uses the RS232Control on its form. These functions are the only communications functions that are required when implementing a custom native-mode RS-232 interface.

Functions for the RS232Control Object

Name	Type	Arguments	Description
PortOpen	Boolean	(none)	Call the Port Open method of the MSComm control

ReadChar	Boolean	buffer as String	Read a character from the input buffer
RS232Disconnect	Boolean	(none)	Close the RS232 connection
WriteChar	Boolean	ch as String	Write a character to the device
WriteCharString	Boolean	cmd as String	Write a character string to the device

P849 Functions

The functions in this table are part of the P849 driver that comes with the tester. These functions are not part of any object so they are delineated in the driver code. Therefore, only the P849 driver has access to these functions. These functions are written to implement the RDP control scheme. Use of the standard RS-232 functions (from Functions for the RS232Control Object) without adding in RDP control violates the RDP syntax. Note that the RDP-specific functions use the core functions.

Functions for the Default P849 Driver

Name	Type	Arguments	Description
SendRequest	Boolean	ByVal cmd As String, ByRef reply As String, ByVal replyLen	Function to send a "Generic" ASCII request to the equipment and receive a response
SendCommand	Boolean	ByVal command As String	Sends a command to the equipment, use ReadReply to retrieve a response, if any
StartWriteCommand	Boolean	(none)	Determines if handler/prober is listening by initiating the <ENQ>/<ACK>-<NAK> sequence
ReadReply	HandlerResult	reply As String	Reads the reply from a previous request using SendCommand. Not required for SendRequest
readCharString	Integer	ByRef buf As String	Strips out <STX> and <ETX> from an incoming message from handler/prober

<	>	
	2015 Teradyne, Inc. All rights reserved.	

hprs232.4.11

<

>

RS-232 Interface : Important Notes

Important Notes

The major advantage of the serial interface, security, is also its major disadvantage. Due to the protocol overhead (which provides the security) and the baud rate limitation, the serial interface is the slowest interface available on the tester. Currently, the tester can send serial signals at a maximum baud rate of 38400. Most handlers and probers can only communicate at 9600 baud with a few capable of 19200 baud and virtually none at 38400 baud.

In addition to the baud rate limitation, RDP requires significant security overhead. This table lists the minimum theoretical communications time over the serial interface for various baud rates and parallel test modes. Text in the table that is displayed lighter indicates that the option is not currently available and is meant for reference purposes. If RDP is not used, the overhead can be less (or greater, depending on the application) but still baud rate limited.

Serial Communications RDP Minimum Theoretical Communications Times¹

Transmission Rate	Single Site	Dual Site	Quad Site	Octal Site	16 Site	32 Site	64 Site
9600 baud	27 ms	33 ms	47 ms	73 ms	127 ms	233 ms	447 ms
19200 baud	13 ms	17 ms	23 ms	37 ms	63 ms	117 ms	223 ms
38400 baud	7 ms	8 ms	12 ms	18 ms	32 ms	58 ms	112 ms
57600 baud	4 ms	6 ms	8 ms	12 ms	21 ms	39 ms	74 ms

¹ Minimum time does not include: test program run time, instrument setup and reset, full datalog time, or intercommunications delays. An intercommunications delay is the time that a handler,

prober, or tester takes between each character transmission. This data is approximate and may vary slightly per tester.

< >		
	2015 Teradyne, Inc. All rights reserved.	

hpsimul



IG-XL Simulators

IG-XL Simulators

When commands are received in simulation mode, rather than sending them over a data bus (which may not exist and would cause errors), an event is raised that can be caught by an event handler in the driver form. The event handler code can process the command and return a response through the communications control, which to the driver code is indistinguishable from a real hardware response. If a realistic sequence of responses is coded into the simulator code, a high degree of testing and debugging of driver code can be achieved before ever connecting to the equipment. Most Teradyne-supplied drivers include a prewritten simulator with the driver.

Simulator Usage

The [EquipmentInterface](#) Sim checkbox (if selected to be visible by the [SimVisible](#) property) allows you to enter simulation mode for the selected handler/prober driver. As previously noted, the simulation mode sends and receives stimulus that the driver would expect from the handler or prober. You can use the simulator with or without a test program loaded.

Simulator Flow

The simulator flow follows the flow of the driver since the simulator waits for opportunities to provide information. The simulator must be involved in two major areas of the test cycle flow: test start and binning. All other areas are optional, although a quality simulator provides one-to-one interaction with each aspect of the driver.

The simulator is engaged or disengaged when you select the [Sim](#) checkbox. Selecting or deselecting the [Sim](#) checkbox generates an event that executes the [SetSimulatorMode\(\)](#) function in the handler/prober driver code. If the function is not present, an error is displayed and simulation is not possible.

Typically the [SetSimulatorMode\(\)](#) function simply sets variables, but, most importantly, it sets the [SimulatorMode](#) of the communications control. Currently, only the [RS-232 Control](#) and the [GPIO Control](#) have a [SimulatorMode](#) property that will engage the simulator code. To ensure that the

latest simulator status and setup is used for the simulated production cycle, click the Connect button in the EquipmentInterface to call the SetSimulatorMode() function before calling the Connect() function in the driver.

When the simulator is engaged and the communications control is initialized, the Initialize event is generated which calls the Initialize function for the particular control. From that point forward, whenever any communications is sent to or received from the handler or prober it is actually to or from the simulator inside the driver. The simulator-only events for the communications control route the communications to the Read and Write functions instead of to the physical hardware. For details about the simulator events and functions, see the section on the particular control.

Simulator Design

Since a simulator is only as accurate or useful as designed, make sure the simulator mimics the handler or prober communication as closely as possible. Realistically, this is not possible, but a close approximation to real-world conditions is. Using the manufacturer’s documentation, the simulator can be primed with the same responses and messages that the equipment would use. The best strategy is to implement a simulator that can respond to the most common conditions but has a catch-all routine to ensure that no condition is unexpected. For the test start area of the test cycle, it is strategic to make the simulator provide all possible conditions (including number of sites, retest, and errors) so that the driver can be thoroughly exercised. In the binning area, since no response is usually required (this varies by handler or prober manufacturer), the simulator only has to accept the data.

<

>

2015 Teradyne, Inc. All rights reserved.